

SoK of Private Deep Neural Network Inference with Approximate Fully Homomorphic Encryption

ZAIRA PINDADO, Barcelona Supercomputing Center (BSC), Spain

THOMAS SPENDLHOFFER, Barcelona Supercomputing Center (BSC), Spain

MOHAMED ALLAM, Barcelona Supercomputing Center (BSC), Spain

PRIYAM MEHTA, Barcelona Supercomputing Center (BSC), Spain

LENA MARTENS, Barcelona Supercomputing Center (BSC), Spain

ANTONIO J. PEÑA, Barcelona Supercomputing Center (BSC), Spain

Deep neural networks (DNNs), a hot topic in this decade, are already solving many practical problems previously unchallenged. There are clear use cases of strong requirements for privacy protection in DNN models and input data. Fully Homomorphic Encryption (FHE) schemes provide privacy by enabling operations upon encrypted data with post-quantum security, at the expense of vast data size increase. Overwhelming execution times and memory sizes currently limit DNN inference with FHE to severely reduced models and datasets. In this article, we thoroughly review the state of the art and the state of the practice around this topic, and identify the current challenges remaining to enable efficient DNN inference with FHE in production-sized use cases, along with the most promising trends to address them. Advancing upon previous review articles in the literature, our focus is specifically on deep learning inference on top of high-performance hardware. From our analysis, we set what we consider to be an ideal private inference system for DNNs, capturing notions already present in the literature and generalizing them.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Computer systems organization** → *Neural networks*; • **Security and privacy** → *Public key encryption*; • **Software and its engineering** → Software performance.

Additional Key Words and Phrases: Fully Homomorphic Encryption, Privacy-Preserving Machine Learning, Private Inference

ACM Reference Format:

Zaira Pindado, Thomas Spendlhofer, Mohamed Allam, Priyam Mehta, Lena Martens, and Antonio J. Peña. 2026. SoK of Private Deep Neural Network Inference with Approximate Fully Homomorphic Encryption. 1, 1 (January 2026), 42 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Deep learning (DL), implemented by deep neural networks (DNNs), has been a hot topic for more than a decade. DNNs are already solving practical problems previously unchallenged such as face recognition, while others such as autonomous driving are presenting clear signs of maturity. In order for a DNN to be able to learn functions that are more complex and/or attain higher precision, DNNs are becoming increasingly larger. Clear examples are large language models (LLMs), with, for example, GPT-3, featuring 175 billion parameters, and GPT-4 estimated to have broken the trillion parameter boundary.

Authors' Contact Information: [Zaira Pindado](mailto:zaira.pindado@bsc.es), zaira.pindado@bsc.es, Barcelona Supercomputing Center (BSC), Barcelona, Spain; [Thomas Spendlhofer](mailto:tspendlh@bsc.es), Barcelona Supercomputing Center (BSC), Barcelona, Spain, tspendlh@bsc.es; [Mohamed Allam](mailto:mohamed.allam@bsc.es), Barcelona Supercomputing Center (BSC), Barcelona, Spain, mohamed.allam@bsc.es; [Priyam Mehta](mailto:priyam.mehta@bsc.es), Barcelona Supercomputing Center (BSC), Barcelona, Spain, priyam.mehta@bsc.es; [Lena Martens](mailto:lena.martens@bsc.es), Barcelona Supercomputing Center (BSC), Barcelona, Spain, lena.martens@bsc.es; [Antonio J. Peña](mailto:antonio.pena@bsc.es), Barcelona Supercomputing Center (BSC), Barcelona, Spain, antonio.pena@bsc.es.

2026. Manuscript submitted to ACM

Manuscript submitted to ACM

1

There are clear use cases for DNNs where the data provided to the network is sensitive or confidential, such as intellectual property or sensitive data protection. In these scenarios, privacy-preserving technologies, including cryptographic primitives such as fully homomorphic encryption (FHE) [55], oblivious transfer [137], or two-party computation [20, 164], are required. At a first glance, cryptography, which aims to prevent information leakage, and DL, that aims to extract information from the data, seem to be in direct opposition. However, FHE enables performing computations directly on the encrypted data and obtain the (encrypted) result of the operation without the need for decryption. The encrypted result is correct as if it had been produced by the traditional method (decryption, computation, and encryption) and may be decrypted regularly using the secret key. The owner of the data has control over it and the DNN may be evaluated in a private fashion.

Research in FHE has seen rapid growth in recent years. Efforts in the field have focused on improving the efficiency of cryptographic operations by algorithmic design [27, 34] and by designing dedicated hardware accelerators [2, 74]. It is now also possible to easily test and build prototype implementations due to the appearance of many open-source FHE software libraries [1, 5, 166]. This rapid development and interest in FHE has made the technology attractive as a tool to potentially provide privacy in many production scenarios, in particular, in DNN inference.

A private DNN inference system constructed with FHE considers a client who owns the data and a server which owns the DNN model, already pre-trained and compatible with homomorphic inference (i.e., not including non-linear functions such as ReLU or SoftMax). The client encrypts the data using FHE to maintain its privacy and sends it to the server. The server then executes the NN homomorphically over the ciphertext and returns the result to the client. Note that the server does not need to reveal its DNN model or weights. This solution is considered to be non-interactive, since just the first message with the data and the final message with the inference result are exchanged. An example of a use case in the medical sector is a DNN (held by the server) which is trained to distinguish between cancer and non-cancer samples of MRI images, and a doctor (the client) that uses it without disclosing any sensitive information from their patients (including their MRI files).

We aim to define a private DNN inference system that captures all the properties/capacities that will make it practical and ready to use while maintaining the privacy of the data and the model. Ideally, for this purpose, it must be scalable, compatible with FHE, general, so any type of data and model may be processed, and with the necessary backends for efficient executions over the available hardware. Constructing such systems is far from an easy task, since practical homomorphic encryption implementations emerged relatively recently and FHE is not exempt from drawbacks that render it currently impractical in many scenarios.

In general, FHE requires the selection of several parameters: ensuring a high security level and sufficient operational depth requires large memory and runtime overhead. This overhead varies depending on the security parameters, but 100x for compute and 100x–10,000x for data size are usual for common parameters. Even when leveraging bootstrapping, a costly procedure that enables operating over the initial depth, there is a limit in the number of operations that may be performed. This is because, as the depth of operations increases, the polynomial degree also increases in order to maintain the same level of security. Additionally, there are challenges when we apply FHE in DNNs, where the data is composed of real numbers. Currently, no FHE scheme supports floating-point arithmetic and may efficiently perform computations across all types of layers commonly used in DNN models. Those that are compatible with all types of operations in the layers do not support real numbers and require to perform the expensive process of quantization. On the other hand, those that do not require quantization require the use of approximations or functional bootstrapping, which are as well computationally expensive. In our work we focus on the CKKS scheme, because of its suitability to

work with real numbers by definition and its SIMD capacity (see Section 4 for a justification); however, we have to deal with the fact that in this scheme only additions and multiplications may be evaluated directly.

Currently, limited RAM sizes and very long inference times make homomorphically encrypted deep learning inference still impractical for many applications. The size of the data increases fiercely when encrypted, leading to a considerably higher computation time when compared to the unencrypted execution. Fortunately, many efforts have advanced the field in this aspect since Gentry [55] introduced the first practical FHE implementation around the early 2010s. These include improved FHE schemes [33, 38], but also software optimizations and hardware specializations that decrease (1) the data footprint and (2) execution time. On top of that, several efforts have dealt specifically with DL-related issues, ranging from HE-friendly networks to HE inference frameworks/compilers/engines [107].

This work is a systematization of knowledge that sets a general definition of a *Private Inference Engine operating FHE*, a notion advanced upon simpler concepts found in the literature such as FHE/DNN compilers or prediction systems. As a difference, we strive for a general (i.e., DNN-model-oblivious), scalable (i.e., able to execute today’s production-ready DNNs), and agile system (i.e., able to execute inference tasks within a latency considered appropriate by the users of the specific use case). Additionally, we perform a thorough analysis of the suitability of FHE when leveraged for privacy in DNN inference, as well as a unique comparison between TFHE and CKKS schemes from the position set in this work.

In summary, the main contribution of this article is our unique analysis of the techniques and approaches required to construct the described systems, by exploring current challenges along with the state-of-the-art solutions in the literature trying to solve or circumvent them. To accomplish this end, we first review techniques to integrate bootstrapping into a NN, function approximation methods, linear operations such as convolutions, and matrix multiplication optimization approaches—those already present in implementations, but also future directions guided by the most recently published theoretical works. In addition, we review parallelism techniques to deal, for instance, with data that is larger than the maximum ciphertext size (by splitting it in different ciphertexts, which are processed in parallel). Next, we review hardware acceleration techniques, including GPU, FPGA, and ASIC-based platforms. On the input data front, we discuss works that process images, but also other inputs such as audio or text, including not only traditional NN models, but also recent transformer architectures. We end our analysis comparing the performance of the different solutions that we review—noting a gap in the literature with respect to experimental evaluations focused on scalability and parallel inference—before discussing our conclusions.

2 Background

In this section, we first establish the notation we use in the rest of the manuscript. Next, we provide the necessary background on deep neural networks and cryptography to understand the rest of the work.

2.1 Notation

We express vectors ($\mathbf{v}$), matrices ($\mathbf{M}$), and tensors in bold. Vectors are noted in lower-case, while for matrices and tensors we leverage capital letters. Given $\mathbf{v}$, v_i denotes either the i -th scalar component of the vector or the i -th element of an ordered finite set of vectors. For positive integers q, N , we denote by $\mathbb{Z}$ the set of integers; $\mathbb{Z}_q$ as the set of integers modulo q ; and $\mathbb{Z}_q^*$ as the multiplicative set of integers modulo q . We denote by $\mathbb{C}$ the set of complex numbers.

2.2 Deep Neural Networks

A (deep) neural network or model is composed by many layers of interconnected neurons. We briefly mention the most common types of layers in the following:

Activation: A non-linear function that decides whether the neuron is activated or not. Some examples are “rectified linear unit” (ReLU), “gaussian error linear unit” (GELU), and “sigmoid linear unit” (SiLU).

Batch Normalization: Normalizes the data to zero mean and unit variance, increasing the stability of trainings.

Pooling: Reduces the data size by aggregating information (e.g., by obtaining the maximum or the mean).

Fully Connected Layer: Performs the dot product between the value of the output in the previous layer and the weight of the neuron.

Convolutional Layer: An $n \times n$ layer that produces a dot product among the neighbor values to result in a convolution. Its purpose is to “learn” features extracted from the dataset. It is often used for image classification.

A *convolutional neural network* (CNN) is a class of DNN that contains a convolutional layer.

A *recurrent neural network* (RNN) is a DL architecture that works with sequential data, such as text, DNA sequences, speech, or time series. The outputs of such models depend on previous computations, which are stored in the model’s internal memory, hence exploiting current and past inputs to predict new data instances.

2.3 Fully-Homomorphic Encryption

Fully homomorphic encryption (FHE) schemes enable to perform arbitrary computations over encrypted inputs, i.e., any number of additions and multiplications. The security of FHE schemes is based on the hardness of the learning with errors (LWE) problem [114, 139] and its variants, such as ring LWE (RLWE) [151], all related to various well-known hard lattice problems (post-quantum secure [114]). In his thesis, Gentry also provided a method called *bootstrapping* [55] for constructing general FHE schemes from those that may operate with a limited number of operations (leveled HE schemes). Many constructions have been proposed afterwards, with TFHE and CKKS being among the most modern and widely leveraged by the literature related to the focus of this manuscript. TFHE [38], also referred to as CGGI for its authors, is an exact FHE scheme that enables to compute any boolean gate operation or non-linear function over bits homomorphically. Its security is based on a generalized version of the LWE problems: the Torus LWE. We refer the interested reader to [39, 80] for further details on the inner workings of TFHE. In this paper, we focus on CKKS and discuss this choice when analyzing the suitability for constructing a private inference engine in Section 4.

2.4 The CKKS FHE Scheme

The Cheon-Kim-Kim-Song (CKKS) scheme [35] is an approximate FHE scheme that was introduced in 2017, and it has become one of the most used in practical applications. It was defined as a leveled HE scheme and extended to an FHE scheme in 2018 [33]. This scheme is defined over a cyclotomic ring $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, using RLWE-based encryption. To ensure security, the encryption procedure adds a random element (the noise or error), following a given error distribution, which increases every time a homomorphic operation is performed over a ciphertext. The key difference between CKKS and other schemes such as TFHE, is that during the decryption function the noise is not removed. This makes the decryption structure to be $m + e \approx m$, where m is the encrypted message and e is the noise. If e is sufficiently small compared to m , it will only alter the least significant bits of m , enabling us to work with this approximation if we control the precision, i.e., the number of bits that are not affected by e . This design enables CKKS to be natively compatible to perform approximate homomorphic operations over complex and real numbers, which is an ideal scenario in machine-learning applications, as we discuss in Section 4.

The scheme works with L residue rings $\mathcal{R}_{Q_\ell}$, where $Q_0 = q_0$ is the zero-level modulus. The modulus at level ℓ is defined by $Q_\ell = \prod_{i=0}^{\ell} q_i$, typically $\approx q_0 \Delta^\ell$, where Δ is the scale factor and $q_i \approx \Delta$. The q_i were q in the original scheme

and they are primes of similar size in the RNS version (Section 2.4.1). The scale factor is used to scale the plaintext, i.e., the encoding of the message (Section 2.4.2), which sets the highest precision possible (it will decrease, since each operation will increase the error). Since each encoding includes the scale factor, after multiplying two ciphertexts, the scale factor is squared and a rescaling operation is needed, i.e., to recover the original scaling we divide by q_ℓ that is $\approx \Delta$. This rescaling produces that the ciphertext changes from level ℓ to level $\ell - 1$. We call L the *multiplicative depth*, since it sets the maximum number of multiplications that may be done before the ciphertext needs to be refreshed to continue operating. Once the level Q_0 is reached, we run the bootstrapping procedure, which, in CKKS, extends the modulus back to Q_{L-b} (where b is the levels consumed by bootstrapping).

2.4.1 RNS-CKKS Scheme. The residue number system (RNS) representation is the integer Chinese Remainder Theorem (CRT) representation, where integers in a field $\mathbb{Z}_{Q_L}$ are represented in $\mathbb{Z}_{q_0} \times \dots \times \mathbb{Z}_{q_L}$ as vector of the modulus at each field, where q_i are pairwise co-primes. In RNS-CKKS we use this representation with an additional zero-level modulus q_0 and a chain of same-size prime moduli $q_1, q_2, \dots, q_L$, satisfying $\Delta/q_i \in (1 - 2^{-\eta}, 1 + 2^{-\eta})$ for some precision η . $q_i \equiv 1 \pmod{2N}$ (to ensure a $2N$ -th root of unity exists and support efficient number theoretic transforms (NTT)) for $i = 1, \dots, L$. All polynomial multiplications are performed on ring elements in the polynomial CRT representation where all integer components are represented in the integer CRT basis.

2.4.2 Plaintext Encoding. Plaintext encoding is intended to encode multiple messages in a single ciphertext, known as *packing*, which enables parallel processing in a *single instruction, multiple data* (SIMD) manner, a technique where a single operation is applied to multiple data elements simultaneously. An arbitrary polynomial representing the messages $m(X) \in \mathcal{R}$ is transformed into a complex vector $\mathbf{z}$ such that $\mathbf{z} = \pi \circ \sigma(m(x)) \in \mathbb{C}^{N/2}$, where π is a projection and σ is the canonical embedding map (see details in the original work [35]). We call $\mathbf{z}$ the *slot or evaluation representation*. After decryption, the inverse operation should be done to obtain the original message $m(X)$ in *coefficient representation*.

2.4.3 Bootstrapping. The goal of the bootstrapping process is to raise the ciphertext to a higher level to continue operating. The maximum achievable level is $L - b > 0$, where b is typically between 13–15 [49]. After bootstrapping, depth- $L - b$ computations may be performed on the ciphertext. The basic idea of bootstrapping is to run the decryption algorithm homomorphically. Assume ct is a correct ciphertext for a certain message $m(X)$ under ciphertext modulus Q_0 , i.e., $m(X) = [\langle \text{ct}, \text{sk} \rangle]_{Q_0}$. The goal is to produce another ciphertext ct' for the same message with a bigger modulus $Q_{L-b} \gg Q_0$, such that $m(X) \approx [\langle \text{ct}', \text{sk} \rangle]_{Q_{L-b}}$. This process includes the following steps, given a ciphertext $\text{ct} \in \mathcal{R}_{Q_0}^2$ in modulus Q_0 , that could be decrypted correctly to some message $m(X)$: (1) Modulus raise, which obtains a ciphertext $\text{ct}' \in \mathcal{R}_{Q_{L-b}}^2$ encrypting the message $m + k \cdot Q_0$; (2) coefficients to slots (CtS), to change to representation to slots; (3) the modulus reduction operation, to remove overflows, $k \cdot Q_0$, introduced in step (1); and (4) slots to coefficients (StC). The most common strategy for step (3) is to approximate it by polynomials. There are many works where the approximation of the modular reduction is improved [32, 64]; others, instead, focus on optimizing one of the most costly operations in bootstrapping, the key-switching operation [27]. Given a ciphertext that has been transformed to correspond to a different secret key, this operation reconstructs a ciphertext of the same plaintext message under the original secret key. Another line of research leverages a different approach for emulating decryption homomorphically: rather than homomorphically evaluating the modulus operation, it subtracts the $k \cdot Q_0$ part from the message [2, 85].

Something inherent in approximating with polynomials is that this approximation may fail. If a single coefficient would fall outside of the approximation interval, then the bootstrapping procedure would fail and return unusable

values. This is measured by the so-called *failure probability* of bootstrapping, and reducing this probability has been studied in the literature [27].

The notion of *functional or programmable bootstrapping* [40] refers to a bootstrapping that does not just enable to continue operating with the ciphertext, but it also evaluates a function over the plaintext. The resulting ciphertext is the encryption of the function of the plaintext which has increased its capacity to continue being operated. This functionality has been recently added to CKKS [10, 91].

3 Private Inference Engines

In this section, we define our notion of an inference engine and explain the challenges when constructing such a system around FHE. Writing secure applications with FHE is unquestionably a challenging task for programmers without a cryptographic background. First, the homomorphic encryption scheme needs to be instantiated with parameters that are not only secure, but also (as possible) enable efficient calculations. Second, the cleartext data needs to be packed into ciphertexts that support only a limited number of operations, requiring the whole program to be expressed as a sequence of operations on ciphertexts (i.e. homomorphic primitive operations). On top of that, each ciphertext contains noise that grows with each operation and needs to be managed, creating non-intuitive performance trade-offs.

There have been several efforts aiming to make FHE more accessible by developing tools that provide a high level abstraction, removing the need for programmers to interact with the ciphertexts directly. In their survey, Viand *et al.* [155] collect such tools under the concept of *compiler*, but they remark that the term is used loosely in the context of FHE and some of them function more as interpreters or libraries. Furthermore, even though many of these compilers include optimization for packing data into ciphertext (e.g. [23, 44]), specialized implementations for managing the layout of ciphertext tensors in the context of ML sometimes prefer to use the term *framework* (e.g. [4, 49]) instead, similar to unencrypted ML frameworks like PyTorch or JAX. These traditional (i.e. unencrypted) frameworks are supported by a rich software ecosystem, that enables even ML novices to pick a pre-trained model and deploy it (for example on a cloud service) with just a few lines of code.

In order for private inference to become practical in production applications, we consider essential for the (encrypted) software stack to attain a similar level of maturity, enabling users to deploy a model without a background in ML or cryptography. In other words, an ideal black-box tool in this scenario would combine the aforementioned FHE functionality with the ease-of-use of current ML platforms. We use the term *private inference engine* to describe such a (software) tool that accepts an arbitrary (pre-trained) model as an input, converts it to an equivalent FHE version, and runs inference on that network for encrypted inputs (without ever having access to the unencrypted data), ideally as efficiently as possible, and performing applicable state-of-the-art optimizations automatically. With regards to FHE functionality, we require this inference engine to support at least the following functionality:

Model Transformation: Convert a cleartext model into the equivalent FHE version, including data layout optimizations and data packing.

Cryptographic Optimization: Determine the optimal parameters and instantiate the underlying FHE schemes as efficiently as possible while maintaining the desired level of security.

Circuit Optimization: Handle noise growth, performing the bootstrapping procedure as appropriate.

Target Deployment: Deploy efficiently on the available hardware (e.g., CPU, GPU, FPGA, ASIC, etc.).

It is well known that combining NNs and FHE is markedly challenging. Thus, for constructing a private inference engine that is user-friendly and amenable for its use in production, we have to deal with several difficulties:

Non-linear Functions: NNs commonly employ activation functions such as ReLU or operators such as max-pooling that are non-linear, while FHE supports only linear function evaluation. There are different approaches that may be used to address this problem: (1) client-aided design [113]; (2) approximation by a polynomial function; (3) replacement of non-linear functions with FHE-friendly (linear) functions; (4) combination with other cryptographic tools, the most commonly used being MPC primitives like secret-sharing, oblivious-transfer, or garbled circuits; and (5) functional bootstrapping.

Scalability: Operations on FHE ciphertexts are much more computationally intensive than for clear data. Thus, scalability for large models and datasets is a primary concern, since encrypted inference may easily last several minutes, hours, or even days, depending on the size of the model. We identify several considerations:

Memory: Ciphertexts, encoded plaintexts (CKKS), and necessary evaluation keys require large amounts of memory (easily starting in the order of 100 GB) that far exceed those of cleartext models.

Parallelism: While parallel processing of ciphertexts is an option, parallelism might be inhibited by the memory requirements. Only a handful of open-source FHE libraries support internal parallelism.

Accelerators: GPU support for FHE has increasingly become available [52, 82, 88], but for many FHE operations, memory access is the main bottleneck [45]; hence, other hardware architectures such as ASICs [146], FPGAs [3, 141], or processing-in-memory (PiM) [78] have been explored.

Error Management: Ciphertexts in FHE inherently contain an error term, which increases with subsequent operations. Managing that error is important in order to obtain a valid result; however, there are trade-offs between error and performance:

Parameters: To sustain accurate results over a sequence of further operations, larger parameters are required.

Bootstrapping: A highly time-consuming but essential operation for large models to maintain practical parameters; thus, the number of bootstraps and their placement pose overly relevant performance implications.

Security: Selecting the proper parameters for an FHE scheme is non-trivial, with standards [26], tools [9], and formulas [95] providing indications on the parameter values that attain a desired security level. However, these parameters also impact performance, so finding a balance between security and computational load is extremely relevant. Another concern is the adversary model, being honest-but-curious (or semi-honest) the most frequently used, which assumes the parties behave honestly (do what they are supposed to do), but they may try to gain information. This model may be improved by adding a proof along the inference result that verifies the operations that have been performed to produce it (verifiable FHE (vFHE) [96]). While some vFHE solutions do not disclose information about the model, these are limited to theory and rarely adopted in practice.

We note that even though various solutions exist that attempt to tackle the above issues and enable automatic private inference based on NN models or tensor descriptions (e.g. Orion [49], ANT-ACE [107], HELayers [4]), current implementations are still limited to certain architectures or NN layer types. Section 5 provides an in-depth analysis of these existing frameworks/compiler alongside other solutions that have been proposed in the literature.

4 Suitability for Private NN Inference

In this section, we analyze the suitability for constructing an inference engine with FHE. First, we justify our choice of using stand-alone FHE solutions over hybrid solutions, which use FHE in combination with other privacy-preserving technologies. Next, we analyze the suitability of the two most used FHE schemes in private NN inference: TFHE and CKKS. While TFHE features the fastest bootstrapping, CKKS is the most suitable to work with real numbers.

4.1 Fully-FHE Solution for Private NN Inference

The main advantages of building private NN inference using FHE over other privacy techniques like two-party computation or oblivious transfer (OT)-based protocols, are that it offers minimum communication requirements and full privacy over the data and the model. OT is typically used as a building block for interactive computation protocols in MPC and it enables the receiver to obliviously select one of the sending party’s inputs. The sender does not learn the receiver’s choice and the receiver does not learn the non-selected inputs. In DNN inference, it is used for the non-linear layers, but it implies additional communication between the server and the client (and non-negligible computations). Secret sharing, an approach for distributing a secret value via two or more shares, which separately do not reveal any information about the secret, enables evaluating any arithmetic function; however, when working with NNs, this protocol requires one round of interaction per non-linear layer, not just adding additional communication cost but also revealing the number of non-linear layers in the model. Another MPC technique used in DNN inference is garbled circuits, a two-party protocol that aims to compute the value of a publicly known function by two parties over secret inputs. This protocol features a constant number of communication rounds, which is independent from the computed functionality; however, solutions based on garbled circuits leak the structure of the NN, i.e., the number and type of the layers, since that must be known by both parties.

In FHE-based solutions, there is no leakage of information about the NN being run by the server, and the client only learns the result of the inference. With non-interactivity, we avoid additional communication cost, beyond the input from the client and the final output response from the server [165], and also issues such as network latency and high bandwidth usage. Another argument to use an FHE-only solution is that its security relies only on the assumption of the hardness of the LWE problem (or its variants) and the selected parameters, whereas leveraging a hybrid approach implies relying also on the assumptions for the other primitive or its requirements for security (for example, the existence of a simulator in garbled circuits).

Still, we need to find the most suitable FHE scheme for our target. Since DNNs require intensive computations, the multiplicative depth of the circuit is considerably high. Although some works perform inference with BFV [29, 31], the highest performing frameworks leverage either CKKS [49, 81] or TFHE [21, 144], which we analyze next.

4.2 TFHE vs CKKS

There have been previous comparisons between the two schemes for inference, considering relatively small NNs [123]. Others focused on the bootstrapping procedure of different schemes [7]. More recently Krüger *et al.* [98] claimed that CKKS provides the best efficiency for ML applications, given its high precision and throughput. We use the experimental results of the last two cited papers through our exposition. Additionally, we compare the schemes based on their features and their viability for the task of running large NN inference, where we consider performance (time and memory), accuracy, whether the functionalities are available in a public library, and their ease of use (i.e., programmability and ability to use a pre-trained NN).

4.2.1 TFHE. Since TFHE is an exact scheme, there is no accuracy loss when performing FHE operations. A ciphertext represents a single integer and thus, all the operations are on scalar operands. One of the most important advantages of TFHE is that it **supports non-linear operations** through programmable bootstrapping [40] by using an LUT containing the evaluation of an arbitrary function. This functionality maps well with the non-linear functions customarily found in DNNs. Programmable bootstrapping poses relatively low latency for small integers [7]. In addition, it does not add overhead with respect to plain bootstrapping, since the LUT is encoded and used in the blind rotation operation (that is

already part of the TFHE bootstrapping). All other schemes have to modify or add some step into the bootstrapping procedure to provide this functionality. However, a simple operation like integer multiplication may be costly in TFHE, especially if the multiplication result should not wrap around the current plaintext modulus [7]. Another advantage is the relatively **small modulus**. This leads to relatively small keys [115], and hence relatively low bandwidth required to communicate the evaluation keys from the client to the server, but also relatively low memory utilization during execution. There is **software available**, such as concrete-ml [166], which provides a ready-to-use NN API with the support of a large engineering team along with extensive tutorials and documentation.

On the other hand, one of the main disadvantages of TFHE is its **performance**. TFHE is much slower than its CKKS counterpart in most operations [98], except for comparisons. “CKKS takes 31 μ s on average per application-level scalar multiply while the TFHE-rs library takes 2.1s per 32-bit multiply, over 60,000 $\times$ slower, this is almost entirely due to CKKS operations being SIMD” [97]. Then, averaging the time over the number of elements in a vector, CKKS is considerably faster than TFHE. Another noteworthy disadvantage is that TFHE **only supports integers of small bit-width**; hence, only fully quantized neural networks are supported, which implies re-training for most NN applications and requires deep knowledge of ML training. In addition, quantization incurs a loss in accuracy compared to the original operation and it is a time-consuming task which hinders development speed. A previous study concluded that the plaintext space may be efficiently extended to 3-8 bits, whereas extending it to more bits requires doubling the runtime for each additional bit, hence becoming impractical [7].

Recently, TFHE has been extended to be compatible with floating-point arithmetic [22]. In contrast to CKKS, which operates with fixed-point, floating-point enables higher precision for data within large ranges. However, the programmable bootstrapping is slower, the implementation is not publicly available to date, and the performance figures presented are markedly slow compared to those with CKKS or TFHE with integers. E.g., in TFHE with floating-point arithmetic, addition and multiplication last 12 and 82 seconds, respectively [22], while for integer representation and the same precision, 105 milliseconds for addition and 401 milliseconds for multiplication were reported¹ on similar hardware. Our own experiments using the CKKS Lattigo library yield from less than 0.5 milliseconds in a 2^{12} -dimensional ring to 6 milliseconds in a 2^{16} -dimensional ring for the addition of 1,024 scalars, again leveraging a comparable CPU².

Recent works offer improvements over the LUTs used for functional bootstrappings. In Tetris [157], the authors compare the 32-bit LUT used for the sign function evaluation time in CKKS functional bootstrapping of 2,924 ms considering 2^{16} slots, with TFHE lasting 527.54 ms for just one slot. This shows that TFHE LUT evaluation attains lower latency, while CKKS’ offers higher throughput.

4.2.2 CKKS. The CKKS scheme **supports real numbers** natively, since it has the capability of encoding floating-point values by defining a fixed-point arithmetic on them. This maps well with DNN inference, since most NN models use real numbers. Another advantage is performance, mainly due to the single-instruction, multiple-data (**SIMD**) nature of the CKKS encoding. CKKS is able to encrypt an entire vector of floating-point numbers (with the precision being configurable via a scaling factor) into a single ciphertext, instead of encrypting each scalar into a separate ciphertext. Any homomorphic operation on this ciphertext will always operate on the full vector (e.g., a multiplication is an element-wise vector multiplication). If the program to evaluate features a large number of SIMD operations, as in NN inference, the cost of these is amortized over the size of the data dimension. Many inference applications process inputs in batches, which adds another SIMD dimension to the computation. The capacity of SIMD also applies to the

¹zama-benchmarks

²Since we are comparing execution times involving single-threaded server-class CPU executions in terms of orders of magnitude, using the precise same CPU is not required for this comparison.

Scheme	SIMD	Real Numbers	Non-linear Functions	Highest Precision	Software for NN	Software Available
RNS-CKKS	Yes	Yes (fixed-point)	No	53	Yes	Yes
GFB CKKS	Yes	Yes (fixed-point)	Yes	≤ 32	No	Yes
TFHE	No	No (needs quantization)	Yes	32^a	Yes	Yes
Real TFHE	No	Yes (floating-point)	Yes	8	Yes	No

Table 1. Comparing CKKS and TFHE characteristics when using them in NN. Higher precision column considers $N = 2^{16}$. ^a from [59].

bootstrapping procedure, enabling CKKS to bootstrap thousands of numbers at once. Hence, NN inference is well suited to leverage the performance benefits of CKKS. This argument is supported by Krüger *et al.*'s experiments on different homomorphic operations using TFHE and CKKS over $N = 2^{16}$ integers, which showed that CKKS outperforms TFHE in most operations except bootstrapping. There exist several **software frameworks** based on CKKS that are implemented specifically for NN processing, such as Orion [49], which we analyze in Section 5.

One of the disadvantages of CKKS lies on its **programmability**. Given that all operations are defined on full vectors, there are no single-element operations, which makes designing efficient algorithms challenging. For example, to efficiently perform a matrix multiplication, one needs to carefully align the encrypted data through homomorphic rotations, which is a computationally expensive operation that should be used sparingly. Another important disadvantage is that CKKS does **not support non-linear functions**. These may be approximated using polynomials, but to obtain acceptable accuracy, these need to be of considerably high order [54] and leverage many multiplications. This reduces the number of available levels, thus requiring bootstrapping, which is computationally demanding in CKKS. However, some recent results improve CKKS bootstrapping, such as SHIP [34], which decreases the number of levels consumed by bootstrapping from around 13–15 to 6–7 (using the HEaaN framework, which is not open-source). Previous works support arbitrary function evaluations by LUTs [10, 91]. However, only part of the former methodology has been recently released in the OpenFHE implementation (August 2025). Still, the bit-size of the LUTs is relatively small (20 bits maximum), increased by Dumezy *et al.* [48] to 32 bits. On the other hand, CKKS also requires **high memory use**. The ring where CKKS ciphertexts are defined features a large modulus, which results in large evaluation keys and might result in high bandwidth use when communicating these from client to server. Evaluation keys will also be used when performing the FHE operations, requiring higher memory utilization than TFHE during execution.

As already introduced, this is an approximate scheme, which leads to some loss of accuracy when performing FHE operations. Hence, we are not able to recover the exact message that was encrypted, but the bound of the error is controlled and constrained to the least-significant bits. Anyhow, DNN inference is known to be tolerant to some precision loss. CKKS is able to encrypt all 53 mantissa bits of a double precision floating-point value [7], which is below what many datasets used in training/inference require, and more importantly, the precision obtained in NN inference in the clear (without the layer of privacy) is also limited to 53 bits.

4.2.3 Discussion. We summarize the comparison of the RNS variant of CKKS with the TFHE scheme in Table 1, including general functional bootstrapping for CKKS (GFB CKKS) [10] and floating-point arithmetic capabilities in TFHE (Real FHE) [22]. The most clear argument to work with CKKS instead of TFHE in the context of DNN is its performance. The SIMD capability enables to attain the best throughput with good accuracy [7]. CKKS also favors scalability. The critical points of CKKS are the limited accuracy due to the approximate nature and high computational complexity [144]. However, NNs tend to be robust against inaccuracies, even if inference incurs some precision loss, we

are likely to be able to attain comparable end-to-end accuracy, as shown, for example, by Orion [49], which attains comparable classification accuracy using the CKKS scheme for a variety of models and datasets (see Section 6).

Another important point is that NN training in the clear is done over floating-point values. All the FHE schemes except CKKS that are currently available work with integers. Therefore, when training a NN to perform encrypted inference using the TFHE scheme, it is necessary to first quantize the network. Emerging techniques like *Real TFHE* [22], which enable TFHE to deal with real numbers directly, show non-competitive performance with CKKS.

NNs contain non-linear layers that cannot be directly evaluated using CKKS³. So far, these functions have been approximated by a polynomial (in most cases) or replaced by an FHE-friendly operation. These approximations are non-trivial to find for sufficiently high accuracy and might require retraining. However, TFHE only supports small integers and requires quantifying the network. This is non-trivial and implies a loss of accuracy; it also requires retraining or extending the size, which implies higher latency.

In the literature, all papers describing NN inference of large networks (such as Resnet-50) use the CKKS scheme. E.g., in Stoian *et al.* [152], a solution for arbitrary NNs with TFHE, the largest NN features 9 layers, while ResNet models used for CKKS may feature 150 layers [18]. As a matter of fact, the literature contains considerably more FHE-NN implementations using CKKS than TFHE, and these works justify this decision by arguing that CKKS is a more natural fit for NNs because of its inherent SIMD capability, supporting our arguments. This includes all the private inference solutions based on large language models (LLMs) with FHE [143, 168].

It is also worth mentioning existing scheme-switching solutions for which multiple schemes are used [28, 112], where, e.g., TFHE is used for non-linear functions whereas CKKS is leveraged for linear functions. However, scheme-switching performs in the order of N^2 ring operations [2, 34], adding an overhead that is not easily recovered.

5 State of the Practice of CKKS Implementations

In this section, we present existing CKKS-based DNN inference implementations. Early CKKS-based solutions targeted relatively-small DNN models along with the MNIST dataset [75, 123], which is currently considered outdated compared to more complex counterparts such as CIFAR10/100, which are more complex to address (e.g., MNIST contains 60k grayscale images of 28×28 for training, while CIFAR-10 features the same number of images, but colored, with a size of 32×32). In the following, we explore the latest CKKS-based implementations targeting larger DNNs and datasets. We group the analysis following the challenges identified in Section 3, discussing the techniques and different strategies used to address them. A summary is provided in Tables 2, 3 and 4.

5.1 Bootstrapping

The early implementations mentioned above were based on leveled FHE and may only support a predetermined number of sequential multiplications, which limits their scalability. While this fixed multiplicative depth works well for relatively simple and shallow networks, both the runtime and memory overhead increase with the number of levels (i.e., supported multiplications). Since deep networks and more complex architectures (e.g. transformers) require many operations/levels, this overhead becomes prohibitively large in practice. To the best of our knowledge, the largest models (in terms of layer depth and number of parameters) that have been implemented and run solely based on leveled FHE are SqueezeNet [44] and AlexNet [4], using a multiplicative depth of ~ 30 levels.

³At the time of writing, functional bootstrapping in OpenFHE does not provide the functionality for CKKS ciphertexts; a conversion to BFV is needed first.

In order to support a larger number of multiplications, bootstrapping (see Section 2.4.3) needs to be employed to refresh the levels of a ciphertext. While this procedure is still costly in terms of computing time, the originally proposed method [33] has since been improved with regards to algorithmic time complexity [27, 64], attained accuracy [16, 105], and implementation [82], to enable actual production applications.

Lee *et al.* [104] were the first to make use of bootstrapping in CKKS to implement private NN inference for a DNN (ResNet-20). Their paper highlights three relevant trade-offs when integrating bootstrapping into a NN:

- (1) **Security and Efficiency:** Both latency and failure probability of bootstrapping directly depend on the sparsity of the secret key (with hamming weight h) and parameters should be chosen to minimize the risk of failure. However, while a lower h means less probability to fail and reduces latency, it also leads to a lower security, since some attacks are able to exploit sparse keys [42].
- (2) **Levels and Bootstraps:** The number of levels available after bootstrapping decides the number of operations that may be executed before another bootstrap is necessary. Because the complexity of all HE operations depends on the levels, having less levels means cheaper operations, but it also requires more frequent bootstrapping.
- (3) **Complexity and Frequency:** Sparsely packed ciphertexts may be bootstrapped much more efficiently than fully packed versions. Thus, it is beneficial to bootstrap at places where the number of packed values is low, even if the maximum number of operations has not yet been reached. The same argument applies to bootstrapping a single/few versus many ciphertexts.

As an example, Lee *et al.* [104] demonstrate that even though the activation function consumes the most levels, better performance may be attained if the bootstrap is placed before the activation function instead of afterwards. In this way, the convolution before the activation function is executed at a lower level, leading to a reduced latency overall. Despite that, in the final version of their implementation, bootstrapping still accounts for more than 30% of the total time, highlighting the importance of balancing the above trade-offs.

While optimal bootstrap placement has been shown to be an NP-hard problem [125], several papers have attempted to find good solutions via heuristics. FHE-Booster [159] automatically adds bootstrapping to a general FHE task graph while either minimizing *a)* the total number of bootstraps or *b)* the latency of the program. For reducing latency, they employ a score-based heuristic combining three attributes: bootstrapping urgency (i.e. number of remaining operations), number of unsatisfied bootstrapping segments (i.e. additional bootstraps needed), and operation slack (interval between the operations latest and earliest start time). While the latency reduction heuristic yields good results for some networks, its greedy bootstrap placement may result in sub-optimal placement in some cases when compared to *lazy* bootstrapping (i.e. bootstrapping when the number of levels is depleted) [97].

More sophisticated approaches generate a set of “candidate” solutions and then estimate the latency via a cost model to determine the optimal bootstrap placement. To find good candidate solutions, these methods partition the FHE graph at certain points, and then insert bootstrapping either before or after such a partition. For the purpose of partitioning, DaCapo [37] considers the number of *live-outs* (ciphertexts that are used afterwards) to determine points where the dataflow converges, while Fhelipe [97] uses *depth boundaries* and bootstraps either all edges crossing a boundary or none of them. Orion [49], on the other hand, only considers bootstrapping among network layers and introduces *level digraphs* for solving the optimization problem. In their reported results, both DaCapo and Orion use 37 bootstrap operations for ResNet-20, but latency for DaCapo is more than 10× lower, due to running on a GPU. In general, the choice of the underlying FHE library, different parameter settings, and further optimization strategies (implemented within the same work), hinder directly comparing the reported results and evaluate solely the effectiveness of the respective bootstrap

placement. Additionally, these optimizations require the sequence of HE operations to be represented as a graph and are thus tied to a specific interface, since no major standard has emerged yet. Therefore, placing the bootstrap operations manually is still a popular strategy in most private inference implementations.

5.2 Non-linear Functions

Since CKKS does not support non-linear function evaluation, activation functions have to be either approximated, replaced by some HE-friendly function, or be evaluated with functional bootstrapping. Because functional bootstrapping for CKKS is a relatively new method, it has not been applied yet to neural networks and the two prevailing approaches in the literature are (1) accurate approximation of the non-linear function by a high degree polynomial, and (2) replacement of the non-linear function by a low-degree polynomial (e.g., square function [57]). This section provides an overview of both approaches, including a discussion of the associated trade-offs.

5.2.1 Function Approximation. Excluding functional bootstrapping, CKKS only allows two arithmetic operations: addition and multiplication. Consequently, all non-linear functions have to be approximated by polynomials using just add and multiply logic. However, accurate approximation requires high-degree polynomials and their evaluation introduces a large computational overhead, consuming many multiplicative levels. Hence, the goal is to find a polynomial approximation that is accurate enough so that the pre-trained weights of a network may be used directly (i.e., without additional training or fine-tuning), while minimizing the resulting overhead and level consumption.

An early attempt by Chabanne *et al.* [30]—numerical analysis—combined ReLU and batch-normalization to approximate it over the standard normal distribution using polynomials of two, four and six degrees. The resulting polynomials are determined by fitting a function to a set of sampled points and minimizing the mean squared error for a constant polynomial degree. However, subsequent experiments revealed that Chebyshev polynomials outperform both numerical analysis and Taylor series, because they are able to approximate a function in an interval instead of a small neighborhood of a point [67]. This is especially relevant for production applications where the input values are unknown.

Cheon *et al.* [36] proposed the usage of composite polynomials to attain optimal asymptotic complexity for the polynomial evaluation in homomorphic comparison. Further work [101, 103] used such composite polynomials to approximate the sign function and extend this approach to ReLU by using $\text{ReLU}(x) = \frac{1}{2}x(1 + \text{Sign}(x))$. Ultimately, Lee *et al.* [104] used three polynomials of degrees 7, 15, and 27 to implement ReLU in a ResNet-20, demonstrating that this method is sufficiently accurate to replace the activation function in a trained model without the need for re-training. Furthermore, the authors proposed a method for approximating the softmax function $\text{softmax}(x) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$ for each $i = 0, 1, \dots, n$, where the exponential function e^x is approximated in the interval $[-B, B]$ by regarding it as the function $(e^{x/B})^B$ and using a polynomial of degree 12. For evaluating the inverse function inside softmax, the Goldschmidt division method [58] is employed. A similar approach is used by Lu *et al.* [77], who additionally stabilize the softmax evaluation by subtracting the maximum input value from all terms (e.g. $e^{x-x_{\max}}$) and use the Taylor series for approximating the exponentiation as $e^x = \left(1 + \frac{x}{2^r}\right)^{2^r}$ for $x \leq 0$ and $r = 6$, an approach that is also used by NEXUS [168] in combination with a quick sum algorithm. Other works usually follow similar techniques, but the exact implementation details might differ. For example, Trycicle [111] proposes estimation of $x_{\max}$ by statistical means and THOR [119] refines the Goldschmidt division algorithm by enhancing the update formulation to speed up convergence.

Regardless of these advances, in non-transformer architectures, the evaluation of the softmax function is often left to the client in many private inference implementations, since it is usually the last operation in the model. This is

advantageous in terms of latency, but it exposes the last layer of the network to the client, making such implementations more vulnerable to attacks (e.g., model inversion) that attempt to extract information from the model.

In addition to softmax, transformer architectures commonly employ the GELU activation function, which is approximated by many libraries (e.g., PyTorch) using $\tanh(\cdot)$. Approximation strategies then focus either on approximating only the $\tanh(x)$ functions or the entire GELU. Rovida and Leporati [143] use different approximations in each layer with polynomials of degree 119 or 59 for GELU approximation and a polynomial of degree 300 for $\tanh$. Other papers employ piecewise polynomials as originally proposed by NEXUS [168] to approximate $\text{GELU}(x)$ over the range $x \in [-60, 60]$ with an average error of less than 10^{-3} . However, determining the interval for the piecewise polynomial may be problematic under encryption. MOAI [169], instead only approximated $\tanh(x)$ by a 23-degree polynomial in the interval $[-20, 10]$ and reported attaining sufficient accuracy in their experiments.

Transformers also employ layer normalization, calculated for a vector $\mathbf{x} \in \mathbb{R}^d$ as $\left[\frac{\gamma(x_i - \mu)}{\sigma} + \beta \right]_{i=1, \dots, d}$, where μ, σ are mean and standard deviation, while γ, β are affine transform parameters. The main challenge lies in the calculation of the reciprocal square root. This is often solved via iterative algorithms such as Newton's method [172] or adapting Goldschmidt's division [118, 119]. Qu and Xu [136] proposed novel methods (Taylor expansion/rational functions) for finding good initial starting values, thus improving convergence. This approach seems to be the current state-of-the-art used by many private transformer architectures [46, 66]. Other solutions in the literature include approximation by Chebyshev polynomials [111] or a combination of both polynomial approximation and iterative algorithms [140].

While these functions are designed as a drop-in replacement without requiring re-training, they still feature a set of hyperparameters that need to be configured in order to yield high-precision approximations. The expected input range is a major concern, since it decides the resulting accuracy of the approximation. For example, Chebyshev polynomials are defined in the range $[-1, 1]$ and therefore require the inputs to be normalized (e.g. via min-max normalization). If the necessary normalization parameters are not available from the trained model, these may be estimated by running a calibration pass on the unencrypted model with a representative set of input data. Through this calibration process, the normalization parameters may be estimated either on a per-network [86, 100] or a per-layer [49] basis. Rovida and Leporati [143] extended this approach to layer normalization and precompute the mean and standard deviation based on these observations. This simplifies computations significantly and did not induce accuracy loss.

Following these approaches, several DNN architectures (e.g. ResNet and BERT) have been successfully implemented [100, 168]; however, due to the high degree of the employed polynomials, evaluation of the activation function consumes many levels. Consequently, these implementations require frequent bootstrapping, resulting in an increased latency. On the other hand, since the activation function is only approximated during private inference, this method enables the usage of pre-trained models without any special requirements on the training process. This is especially beneficial for large models, since, for example, training an LLM anew may well be prohibitively expensive.

5.2.2 Function Replacement. The goal of this method is to design FHE-friendly NNs that replace non-linear functions with low-degree polynomials that consume less levels than the high-degree approximations from the previous section. This results in architectures that require less multiplicative depth, and thus the number of levels and/or bootstrap operations may be reduced considerably, significantly improving inference latency. A well-known low-degree polynomial replacement for ReLUs is the square function $f(x) = x^2$ [57]. However, especially if the number of layers is large, it suffers from the *escaping activation* problem [54], where the activation values escape the well-behaving range, leading to either exploding values or poor approximations.

Other studies approximate ReLU (for a constant degree D) using polynomial regression to minimize the error between the approximation and the target function f (over the range $[-a, a]$). For the ReLU function, the optimal coefficients $w_0, w_1, \dots, w_D$ may be found by minimizing:

$$E(w) = \int_{-a}^a \left(\sum_{d=0}^D w_d x^d - \text{ReLU}(x) \right)^2 dx.$$

One example for such a ReLU replacement is the quadratic polynomial $f(x) = 0.00047x^2 + 0.5x$ [153]. While these functions may attain higher accuracy than the square approximation, they still suffer from escaping activations. Thus, to mitigate the potential unbounded range (i.e., the escaping activation problem), there is a need to constrain the pre-activation values by enforcing an upper/lower bound. This may be attained by min-max normalization, where the minimum and maximum values are observed and adapted during training [54]. While this approach enables stable training and generally attains good accuracy, it also implies re-training the whole network.

In a similar fashion, Park *et al.* [128] proposed an accuracy preserving low-degree polynomial activation function (AESPA), replacing both ReLU and batch normalization layers with the composition of orthogonal basis polynomials and basis-wise batch normalization. The resulting approximation is a quadratic function computed as

$$f(x) = \alpha \sum_{i=0}^d \hat{f}_i \frac{h_i(x) - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where μ and σ are the mean and standard deviation of the batch, $\hat{f}_i$ is the i -th Hermite coefficient of ReLU

and $h_i(x)$ is the normalized Hermite polynomial. The reported performance results suggest that using AESPA instead of ReLU is able to retain the high accuracy of standard ML models across a number of different networks and datasets.

All approaches discussed so far aim to find a good polynomial approximation for ReLU and then use it as replacement throughout the whole network. However, the restriction to low-degree polynomials means that it is usually not sufficiently accurate to work with pre-trained parameters. However, even with retraining, the (now) linear activation function often results in lower accuracy when compared to the original variant. To address this issue, Baruch *et al.* [19] proposed a fine-tuning approach that uses different activation functions per layer, without necessarily approximating ReLU. The authors provided a methodology for modifying a NN architecture during training in order to produce an HE-friendly model with similar prediction accuracy as the original. They use a DNN to train the coefficients of the polynomials for each activation layer [160]. They refer to this technique as *trainable* activation, but in previous studies it is also known as *parametric* or *adaptive polynomial* activation. At its core, the trainable activation uses a polynomial of the form $p(x) = ax^2 + bx$, where a and b are coefficients which are trained individually per layer. Since applying such a significant change to a complex model without adopting the model weights first might lead to a steep drop in accuracy, they introduce a *smooth-transition* policy where the weighted activation function at epoch e (one epoch equals one iteration over the whole training data) is calculated as $w(x, \lambda_e) = (1 - \lambda_e) \cdot \text{ReLU}(x) + \lambda_e \cdot p(x)$. By using the ratio parameter λ_e , the transition from ReLU to the polynomial approximation advances gradually, preventing the loss in accuracy. In addition, their paper employs Knowledge Distillation (KD) to strengthen the new model. This process enables a knowledge transfer from a stronger “teacher” model to a weaker (and usually smaller) “student” model. Considering that the replacement of ReLU with a polynomial activation weakens the model, they regard the original model as the *teacher*, that helps in training the HE-friendly *student* model. Using this methodology, Aharoni *et al.* [4] trained an HE-friendly version of AlexNet (without bootstrapping) that may be evaluated in under five minutes.

AutoFHE [14] follows a similar approach but allows polynomials of varying degrees, while Zhang *et al.* [170] train square polynomials for each element of the input data, extending the layer-wise approach to an element-wise one.

Replacement methods for the softmax function usually aim to eliminate the division operation, removing the need to approximate the inverse and improving numerical stability. Powerformer [127] reduced the division to a constant term using a method similar to batch normalization and approximated the exponential function by a Rectifier-Power (RP) function using ReLU. Other works [66, 140] proposed using the sigmoid attention mechanism or Gaussian kernel attention instead, both of which require only the exponential function. On the other hand, Al Badawi *et al.* [6] used minimax approximation to identify a good softmax approximation with low degree as $\text{softmax}(x) = \frac{1}{8}x^2 + \frac{1}{2}x + \frac{1}{4}$.

With respect to GELU, replacement methods include composite polynomials (31 and 27 degrees for $\text{GELU}(x)$), two 15-degree polynomials for $\tanh(x)$ [119] or using a different activation function such as ReLU [127] or SiLU [66].

All of these function replacement strategies may lead to a significant reduction in the number of necessary bootstraps, enabling faster inference. As a drawback, this approach requires specialized models that are explicitly trained with the goal of enabling HE-friendly inference. Therefore, this method is not prone to scale for large models such as LLMs, which require vast amounts of resources for training.

It should also be mentioned that, apart from the activations, function replacement is commonly employed to address other nonlinearities such as max-pooling in CNNs. As denoted in CryptoNets [144], the maximum may be approximated with $d = 1$ as a simple sum from the relation $\max(x_1, \dots, x_n) = \lim_{d \rightarrow \infty} (\sum_i x_i^d)^{1/d}$. Consequently, most encrypted inference works replaced max-pooling with (scaled) average-pooling [133] and when considering only CKKS and DNNs, this replacement was used by most papers in Table 3 (where the original architecture contained max-pooling). An alternate is proposed by Guilherme and Wakaha [147], who consider the *max* function as $\max(a, b) = \frac{a+b+\sqrt{(a-b)^2}}{2}$ and use a polynomial approximation of the *sqrt* function in their evaluation.

5.3 Linear Operations

In CKKS, each plaintext or ciphertext corresponds to a vector of elements, and the scheme supports element-wise addition and multiplication among these, as well as rotate/shift operations on single vectors. As a consequence, operations that require single-element access are not straightforward and must be implemented via reduction/replication of vector elements. For example, forming the *dot-product* between two vectors requires one (element-wise) multiplication followed by a summation of all the elements inside the resulting vector (inner sum). This summation may be computed in $O(\log n)$ operations as a sequence of rotations and additions, as illustrated in Figure 1b. However, since each rotation in CKKS is followed by an expensive key-switching procedure, a major design goal when developing more complex algorithms (such as matrix-vector multiplications or convolutions) is to keep the number of required rotations low.

5.3.1 Matrix Multiplication. We consider the matrix-vector multiplication $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{x}$ and $\mathbf{b}$ are encrypted column vectors, and the matrix $\mathbf{A}$ may either be encrypted or in plaintext. When $\mathbf{A}$ is unencrypted, changes in the representation (e.g., column- or row-major) may be easily computed from the underlying data, while format changes for an encrypted matrix require costly data movement techniques, rendering layout selection as a relevant consideration. E.g., if $\mathbf{A}$ is represented in row-major format, naive computation of the matrix-vector product requires $O(n)$ multiplications, inner sums, and masked additions. On the other hand, if the matrix is stored in diagonal order (see Figure 1a), the expensive inner sums can be replaced by single rotations.

If the matrix is available in plaintext format, the number of ciphertext rotations may be reduced to $O(\sqrt{n})$ by using the baby-step giant-step (BSGS) algorithm [61]. As depicted in Figure 1c, the algorithm uses computationally inexpensive

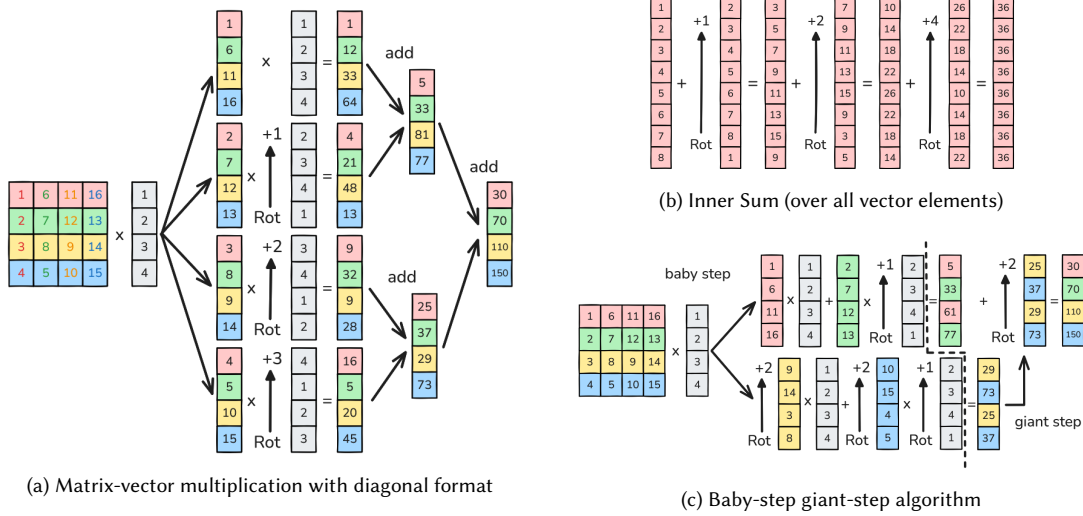


Fig. 1. Linearar Operations in CKKS.

rotations of the input matrix A to reduce the number of ciphertext rotations. Additionally, all rotations during the baby step are used multiple times and may be precomputed from the same ciphertext, which enables an optimization technique called *hoisting* [62]. The key idea behind this concept is that certain computations are independent from the rotation offset and may be shared among multiple rotations of the same ciphertext. A similar concept is used by Bossuat *et al.* [27] to apply a second layer of hoisting, leading to a *double-hoisted* approach, which is currently state-of-the-practice in many cryptographic libraries.

While the above matrix-vector multiplication algorithms may naturally be extended to matrix-matrix multiplication, $AB = C$, by encoding each column of B into an individual ciphertext, this usually results in under-utilization of the available ciphertext slots (unless the dimensions are large). To avoid this issue, Rovida and Leporati [143] introduced a “wrapping” procedure that combines n results from individual matrix-vector multiplications into a single (densely packed) ciphertext. BOLT [126] addressed the packing issue by interpreting each column of C as a linear combination of the columns of A multiplied by scaling factors from B (when B is available as plaintext) or as an outer product ($\otimes$) between the columns A_i and rows B_i , such that $AB = \sum_{i=0}^{i < n} A_i \otimes B_i$. ENSI [66], on the other hand, used the same approach, but if one of the matrices is available in plaintext, it is implemented as a summation (i.e., without multiplications). If both matrices are encrypted, they proposed a novel extract-and-replicate mechanism to obtain the scaling factor from the encrypted matrix. Similarly, Jiang *et al.* [75] denoted the matrix multiplication as a component-wise product ($\odot$) $AB = \sum_{i=0}^{i < n} A_j \odot B_j$ for some matrices A_j and B_j that represent specific permutations of A and B . Combining these formulations with different packing strategies (e.g. row/column/diagonal) in order to optimize the number of rotations, several works have implemented transformer based architectures for private inference [168, 169]. Rho *et al.* [140] followed the same procedure but proposed LoRA (low-rank adaption) [69] to reduce the size of the matrices.

On the other hand, some works aim to benefit from fast linear algebra libraries (e.g. BLAS) to accelerate computations for both *plaintext-ciphertext* [17] and *ciphertext-ciphertext* [130] matrix multiplications, reducing these operations to plaintext-plaintext matrix multiplication, similarly to Gentry-Lee [56]. While initial results are promising, these algorithms have not yet been applied in the context of NN inference.

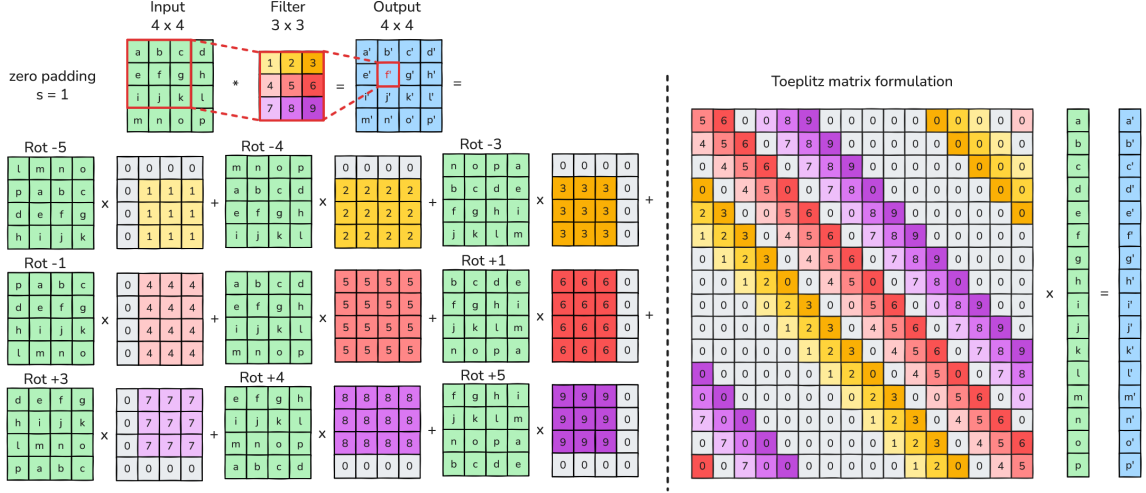


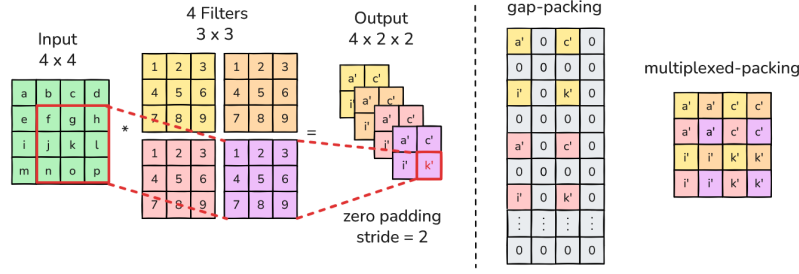
Fig. 2. Convolution with $C_i = C_o = 1$ (left: Gazelle [84], right: Toeplitz matrix [71]).

5.3.2 Convolution. The convolution operation obtains a tensor $\mathbf{I} \in \mathbb{R}^{C_i \times W_i \times H_i}$ (where C_i is the number of channels and W_i, H_i refer to the width and height, respectively) as an input and produces an output tensor $\mathbf{O} \in \mathbb{R}^{C_o \times W_o \times H_o}$. For each channel in C_o the convolution slides a filter bank consisting of $C_i \times F_w \times F_h$ weights over the inputs producing one element of the output tensor in each step (see Figure 2, top left part). Convolutional neural networks (CNNs) commonly employ zero padding to preserve the spatial dimensions of the input (i.e., $W_i = W_o$ and $H_i = H_o$), which we will assume as the default setting in this section. Furthermore, convolutional layers also specify an additional stride parameter s , which denotes the granularity at which the filters are applied.

In the context of FHE, Gazelle [84], proposed an algorithm to perform single-input single-output (SISO) convolution ($C_i = C_o = 1$) that has been followed by many the subsequent works [37, 97]. Their method flattens the input $\mathbf{I}$ into a single ciphertext that is rotated $F_H \cdot F_W$ times, with each rotation being multiplied by a punctured plaintext (encoding a single filter weight). The summation of all partial products then delivers the final result as illustrated in Figure 2. By concatenating multiple channels into one ciphertext (and multiple weights into one plaintext), this approach may even be extended to process more than one channel at a time [84]. Lee *et al.* [104] extended the original SISO algorithm to handle strided convolutions ($s > 1$), observing that the output data for non-strided convolution includes the output data for the strided case and can be extracted by an additional masking step. However, this masking introduces gaps (i.e., zero values) inside the ciphertext, occupying slots that could otherwise be leveraged for useful computation. Furthermore, these gaps increase through consecutive strided convolutions, leading to highly sparsely populated ciphertexts.

Multiplexed packing [92], which maps a tensor of $C^2 \times W \times H$ to a *multiplexed tensor* of size $CW \times CH$ (see Figure 3), may be used in combination with masked rotations and additions to concatenate several sparse ciphertexts into a single densely packed instance. Adapting the convolution operation to this multiplexed format helps reducing the number of ciphertexts (and/or the number of slots within a ciphertext) that need to be bootstrapped, reducing inference time [100].

Despite that, the additional rotations required for re-packing may add significant overhead. Hyphen [87] aimed to minimize this overhead by designing two convolution operations, each producing an output format that matches the input format of the other. Thus, these may be employed in an alternating fashion, reducing the total number of rotations

Fig. 3. Gap-padding versus multiplexed packing for a convolution with $C_0 = 4$ and $s = 2$.

by more than half. Felipe [97] followed a similar idea, but proposed an automated data packing approach instead. It combined a flexible layout representation with a layout assignment algorithm aiming to determine the optimal input format for each layer, to reduce the cost of format conversion among consecutive operations. This is attained by analyzing the NN graph and moving format conversions to earlier points in the graph where they are either cheaper to compute or completely unnecessary (e.g., by finding an input layout that produces output in the desired format). An alternative layout representation based on tensor tiles that support interleaving of dimensions was proposed by HELayers [4] together with a convolution algorithm that requires minimal repacking among consecutive layers. That framework uses an optimizer to determine the optimal packing (i.e. number and dimensions of the tiles) automatically based on some optimization target, for example high throughput, low latency, or minimal memory consumption.

Instead of following the blueprint of Gazelle’s algorithm, slytHERin [71] expressed convolution as matrix–vector multiplication by representing the filter as a *Toeplitz* matrix (depicted on the right side of Figure 2). Hence, this method does not require a specialized implementation for convolution and is able to benefit from optimized matrix–vector algorithms as discussed in the previous section. Additionally, their packing scheme (essentially a row–flattened and transposed tensor) guarantees that the output of this convolution method is a valid input format for any subsequent layer. Orion [49] extended this approach for strided convolution and, by using multiplexed packing, developed an algorithm (*single-shot multiplexed convolution*) able to output a densely packed ciphertext without the need for any masking operation. Their method benefits from the fact that the rows of Toeplitz matrices may be permuted without changing the underlying computation, enabling to re–arrange the weights to produce a desired output format (instead of re–arranging the ciphertext itself).

In contrast, multiple papers (e.g. FALCON [109]) propose to use the *convolution theorem*, by expressing convolution as the product of two Fourier transformed inputs. While this method only requires a single ciphertext multiplication to compute the result, it involves either interaction with the secret key holder or homomorphic evaluation of (inverse) discrete Fourier transforms (DFT/IDFT), introducing additional overhead. Later works remove the need for evaluating the DFT with a novel packing method that directly encodes the input into the *coefficients* of a polynomial [70, 86]. In this case, homomorphic multiplication of two inputs in coefficient encoding results in a convolution of the underlying messages. Kim *et al.* [86] showed that if $I \in \mathbb{Z}^{w \times w}$ and filter $F \in \mathbb{Z}^{k \times k}$ are expressed as two polynomials $I(X)$ and $F(X)$ in $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ with $\max(w^2, k^2) \leq N$, then the $(iw + j)$ -th coefficient of $I(X) \cdot F(X)$ corresponds to $\text{Conv}(I, F)_{i,j}$ if the coefficients are encoded as follows:

$$I(X) = \sum_{0 \leq i, j < w} I_{i,j} \cdot X^{(i-k)w+j} \quad F(X) = \sum_{0 \leq i, j < k} F_{i,j} \cdot X^{wk-(iw+j)} \quad (1)$$

where X^t means $-X^{N+t}$ when $t < 0$. They extend this formulation to sparsely-packed ciphertexts and batched convolution (with batch size B), enabling to reduce the number of rotations to $B - 1$ (slot encoding: $k^2 + B - 2$).

On the other hand, coefficient encoding does not support the element-wise multiplication needed for evaluating an activation function and thus cannot support a NN by itself. As a solution, Kim *et al.* [86] propose to modify the bootstrapping procedure (which internally performs StC and CtS conversions, see Section 2.4.3) in order to combine it with coefficient encoding. In their method, the convolution is evaluated in coefficient domain, right after the StC and before the modulus raising step. Consequently, executing the bootstrap procedure between each convolution and activation function becomes mandatory, possibly leading to an increase in the bootstrapping frequency.

While the proposed batch convolution algorithm may compute convolutions without additional rotations, the output still consists of B separate ciphertexts that need to be repacked into a single ciphertext using rotations and masked additions. NeuJeans [81] proposed a solution to reduce that overhead via *coefficients-in-slots* (CinS) encoding, which represents an intermediate encoding between slot and coefficient encoding. The fundamental property of this encoding is that for each evenly partitioned slice of the vectors $\mathbf{a} = (a_0, a_1, \dots, a_k)$ and $\mathbf{b} = (b_0, b_1, \dots, b_k)$, with a_i and b_i in a subring, multiplication results in a partial local convolution $\langle \mathbf{a} \rangle_{\text{CinS}} \cdot \langle \mathbf{b} \rangle_{\text{CinS}} = \langle \mathbf{a} *_{\text{local}} \mathbf{b} \rangle_{\text{CinS}} = \langle \mathbf{a} *_{\text{part}} b_0, \dots, a_k *_{\text{part}} b_k \rangle_{\text{CinS}}$, where $*_{\text{part}}$ denotes the convolution of two polynomials within a subring of the given polynomial. Additionally, CinS encoding supports cyclic rotation of the sequence of slices, enabling it to combine multiple ciphertexts more efficiently. Nonetheless, since it does not support element-wise multiplication, CinS encoding still needs to be combined with bootstrapping in order to enable evaluation of the activation function.

5.4 Parallelism

There are many forms of parallelism in private inference, depending on the granularity of the operation at which it is applied. At coefficient level, multiple coefficients of the same polynomial may be processed concurrently, while the independent polynomials corresponding to each level of a ciphertext (often referred as *limbs*) may also be scheduled in parallel. At the inference level, one may attempt to pack multiple inputs into a single ciphertext to be processed in parallel or execute operations on many concurrent ciphertexts. While batched inference via packing is a popular strategy to increase the throughput for small inputs (e.g. MNIST), it becomes mostly obsolete for larger datasets since, for example, a typical ImageNet input ($224 \times 224 \times 3$) is already too large to fit inside a ciphertext of polynomial degree $N = 2^{17}$. In those cases, the input needs to be split across several ciphertexts, naturally increasing concurrency.

Even for inputs that fit within a single ciphertext, the matrix-vector multiplication and convolution algorithms discussed in Section 5.3 require multiple temporary ciphertexts that may be processed in parallel. Still, many papers published in the field restrict their experiments to single-threaded evaluation [86, 97] or just mention the number of used threads briefly [87, 116], without conducting experiments around scalability or parallel efficiency.

Some studies were limited to shallow models that allowed for private inference without the need for bootstrapping [43, 171]. Focusing our analysis on inference of relatively large models, which require the bootstrapping procedure, Park *et al.* [129] reported speed-up of 12× when extending a former implementation of ResNet-20 to 64 threads, compared to the original, single-threaded version. Their optimizations, however, might not have been restricted to inference-level parallelism, since their target hardware was a GPU; thus, those optimizations might have also included more granular (limb and/or coefficient level) parallelism inside the homomorphic operations (e.g. bootstrapping) that are discussed in more detail in the following section.

5.5 Hardware Acceleration

While Fast Fourier Transform (FFT) and Number Theoretic Transform (NTT) are commonly employed at the algorithmic level to reduce the overall computational complexity, these operations themselves are difficult to accelerate due to their high computational complexity, extremely intensive memory access, and limited generality [167]. This is especially true for the ciphertext maintenance operations (bootstrapping and key-switching), which require a large number of these transformations. One common approach to address this is to exploit parallelism at the limb or coefficient level. While both work reasonably well for operations such as polynomial addition and multiplication, the situation is more complex when considering the key-switch operation. Key-switching requires both basis conversion and NTT operations, but while NTT generally needs all the coefficients of a single polynomial (and thus, it lends itself to limb-level parallelism), basis conversion may be parallelized over the coefficients, but requires access to data from multiple limbs. Consequently, many FHE accelerators employ either one or a combination of both of these strategies in order to speed-up the core FHE operations. This section focuses on discussing such issues on different hardware platforms. Table 5 provides a summary of some relevant contributions.

5.5.1 CPU-Based Acceleration. Only a few CPU-based acceleration schemes have been proposed and they are somewhat limited in their effect, since the CPU has the least parallel computing resources from all discussed platforms. Boemer *et al.* [24] implemented polynomial modular multiplication and NTT using the Intel Advanced Vector Extensions 512 (AVX512) instruction set and provided it in the form of a Homomorphic Encryption Acceleration Library (HEXL⁴). Since AVX512 instructions may process eight 64-bit integers simultaneously, the acceleration is mainly based on loop optimization (e.g. unrolling) and vectorization (SIMD). While HEXL is able to attain high acceleration when running on a single thread, the overall performance degrades when many threads are used due to heat dissipation constraints [167]. Furthermore, the overall performance is limited by the relatively-slow memory access of CPU-based platforms.

5.5.2 GPU-Based Acceleration. While the main acceleration capabilities of GPUs are based on the large number of parallel computing resources available, these also feature a higher memory bandwidth than traditional CPUs. Hence, assuming an appropriate data storage strategy has been employed, these platforms may be used to speed-up memory-bound kernels due to faster data access. However, GPUs have limited customization flexibility, constraining the innovation of GPU-based acceleration methods when compared to FPGAs or ASICs.

Based on previous work implementing the whole BFV scheme on GPUs, Badawi *et al.* [6] provided PrivFT, a CKKS implementations for GPUs which supports the rescaling procedure. However, the encoding/decoding operations remain on the CPU, since those are not in the critical path for inference applications. Other CKKS implementations on GPUs focus on accelerating specific, time-consuming operations such as multiplication [83, 148] or bootstrapping [82]. While the method employed by Jung *et al.* [83] is based on exposing parallelism and has been implemented for both CPU (multi-threading and AVX512) and GPU, optimizations in the other papers are focused on the GPU architecture and are mostly memory-centric. This is justified by the high data access and the low arithmetic density of many FHE functions, resulting in memory-bound algorithms. The reported speedups in these studies range from twice as fast to over a hundred times faster, depending on the homomorphic function and optimization method. Based on this initial success, a number of studies/libraries has been released dedicated to fully implementing the CKKS scheme on GPUs, such as HEonGPU [124], TensorFHE [53], or WarpDrive [52], with the last two featuring specific optimizations in order to

⁴<https://github.com/IntelLabs/hexl>

leverage tensor cores on modern NVIDIA architectures. Nonetheless, there is no extensive performance comparison for NN inference among GPU implementations, and most solutions limit their evaluation to the ResNet-20 architecture.

Additionally, it is worth mentioning that while the original HEaAN library used in some of the earlier papers [82, 83] is no longer open-source, it is still available⁵ and offers a full GPU implementation of CKKS that has presumably been optimized even further. Despite the extent of these optimizations remaining unknown, the library has been used in a number of studies [18, 81] in order to evaluate private NN inference on GPUs.

5.5.3 ASIC-Based Acceleration. In order to deal with compute and memory intensive FHE operations, the recent wave of ASIC accelerators for CKKS employ hardware–software co-design, wide vector custom functional units, and very large scratch-pad or register files. While early ASICs targeted leveled HE, F1 [145] proposed the first *programmable* FHE accelerator capable of running complete FHE programs via wide-vector custom functional units designed for FHE kernels such as NTT and rotation. It also proposed a novel compiler enabling static scheduling for better orchestration of data movement across the memory hierarchy. However, for high polynomial degrees and large multiplicative depth, this design is still bottlenecked by bandwidth and on-chip memory capacity. CraterLake [146] aimed to address this through a novel key-switching algorithm, which reduces the amount of auxiliary data from GBs to tens of MBs. Subsequent ASICs, BTS [94] and ARK [90], both focus on accelerating the bootstrapping procedure. BTS exploited coefficient-level parallelism in FHE operations by mapping them to its massively parallel 2D grid of processing elements, while ARK used *minimum key-switching* and *on-the-fly* limb extension, to eliminate $\sim 88\%$ of off-chip reads during bootstrapping.

Since CKKS generally operates on large integer values, these are usually broken into smaller parts (e.g., 64 or 32 bits), for example by using an RNS representation. SHARP [89] proposed a short-word (36-bit) hierarchical microarchitecture that localizes NTT communication, introduces novel data-ordering, and short-word functional units. Due to this, SHARP is able to reduce on-chip capacity and power consumption substantially while still retaining performance. On the other hand, Osiris [50] proposed an end-to-end systolic approach to exploit dataflow maps of the baby-step giant-step algorithm and double-hoisted rotations used during matrix-vector multiplication.

Across all these designs, the main limitation is the required data movement (e.g., ciphertexts, evaluation keys, etc.). To mitigate this, ASICs generally propose large on-chip memory and massive amount of functional units, resulting in large monolithic chips. Fabrication of such designs is slightly problematic, increasing the cost of production. To overcome this, Cinnamon [74] and CiFHER [93] designed cluster-based and chiplet-based accelerators, respectively, that provide better cost-efficiency and are highly scalable. Cinnamon also provided a compiler for automatically extracting polynomial-level and limb-level parallelism in a CKKS application in a two-stage compiler pass.

5.5.4 FPGA-Based Acceleration. While GPUs are widely available and offer massive parallelism, their architecture is primarily optimized for floating-point operations, whereas FHE computations require high-throughput modular integer arithmetic. Moreover, the irregular memory access patterns and fine-grained operations in FHE are a poor match for typical GPU execution models, resulting in underutilization of available resources. ASICs, on the other hand, may provide high performance and energy efficiency through tightly customized pipelines and memory hierarchies; however, the development of ASICs incurs remarkably high engineering costs and long design cycles, hindering their suitability for rapid prototyping on evolving cryptographic standards.

FPGAs offer a compelling middle ground. Their reconfigurable logic fabric enables the implementation of customized data paths tailored to modular arithmetic, while offering significantly lower development costs and turnaround times

⁵<https://www.cryptolab.co.kr/en/products-en/heaan-he/>

compared to ASICs. Recent FPGA platforms equipped with high-bandwidth memory (HBM) and relatively larger on-chip memory, such as the Xilinx Alveo U280 and V80, have addressed earlier bandwidth limitations, enabling FPGAs to effectively compete with both GPUs and CPUs in memory-bound workloads such as FHE.

HEAX [141] is one of the first works to target CKKS acceleration on FPGAs. It implements a high-throughput NTT engine and a homomorphic multiplication unit. However, it does not support the bootstrapping procedure. FAB [3], on the other hand, supports the complete CKKS pipeline, including full support for all homomorphic operations. The paper reports a multi-FPGA setting for tasks such as evaluating ResNet. Poseidon [163] also supports all homomorphic operations and implements several hardware and bandwidth optimizations to improve efficiency. On the other hand, HEAP [2] employs a hybrid scheme-switching (HSSw) approach to accelerate bootstrapping, leveraging the TFHE [38] scheme for efficient bootstrapping while utilizing CKKS for the remaining operations. The paper reports both single-FPGA and multi-FPGA configurations on DNN inference tasks for image classification. FAST [162] proposes datapath optimizations for two key operations in bootstrapping: homomorphic linear transformation and polynomial evaluation. These optimizations lead to the reduction of off-chip ciphertext access. Additionally, it proposes a permutation network that may be used to efficiently reorder coefficients during NTT stages and rotation operations.

5.5.5 Memory Technologies. To alleviate memory bandwidth bottlenecks, several studies have proposed implementing NTT on near-memory architectures, mostly based on either Resistive RAM (ReRAM) or SRAM array architectures, not targeting commercially available hardware [60, 106]. There are also works targeting the only commercially available processing-in-memory solution to date (UPMEM PIM) [78, 120]; these works, however, fail to either provide 128-bit security and/or end-to-end execution timings.

Considering that the large memory requirements of encrypted inference have been a common limitation among all of the discussed acceleration approaches, Lloret-Talavera *et al.* [113] proposed leveraging hybrid memory systems of DRAM and persistent memory for private inference on DNNs. They demonstrated that in such settings, data may be efficiently cached inside the DRAM, being able to reduce its required size by 66%, while only incurring a small performance penalty.

5.6 Input Data

Starting from early private inference solutions such as Cryptonets [57] (which classified handwritten digits from the MNIST dataset), considerable research has been focused on improving encrypted image classification. In the context of DNNs, the field has moved to larger and more difficult datasets, such as CIFAR and ImageNet. While the trend seems to evolve toward larger models, that are more powerful in their capabilities, depending on the application, this might not be necessary. CryptoFace [13] has shown that by adapting a tiling approach and developing a specialized network dedicated to face recognition, more efficient solutions may be found to conduct private inference on datasets such as WebFace4M.

Another area that has gained popularity with the rise of LLMs is private inference in natural language processing (NLP). This field contains a variety of tasks such as text classification, information retrieval, spam detection, sentiment analysis, or question answering, among others. PrivFT [6] presented the first CKKS-based solution dedicated to text classification on encrypted data for the fastText model [79]. Since this architecture is relatively shallow, bootstrapping was not employed in their implementation, using only the leveled RNS-CKKS variant instead. More recent approaches, based on word embeddings, usually apply two types of models: one model to obtain a vector representation of the text (i.e., the embedding) and another model that performs the required task (e.g., classification) on the vector data. In the

context of private inference, the first type of model is generally processed on the client side and only the second model works on the encrypted data. Early implementations include recurrent NNs [132] such as long short-term memory (LSTM) and gated recurrent unit (GRU) [73], and reviews of these methods are provided in several surveys [150, 165]. However, following the release and success of ChatGPT, transformer-based architectures have seen widespread adoption in NLP. Transformers present some unique challenges for private inference as, for example, the attention module, which requires 3 matrix multiplications and one softmax evaluation, both of which consume many multiplicative levels. For further discussion on the challenges of private transformers inference we recommend the survey by Li *et al.* [110], whose (non-hybrid) results are extended in our Table 3.

Audio data processing is used for identification and classification of voice and music, sentiment analysis, and other use cases. Some works process audio data with CNNs [41, 51], but use only small networks with a maximum of twelve layers. Similarly, video processing is used for automatic identification, activity detection in video content, and target tracking. Lagesse *et al.* [99] process video data alone using small parameters ($N = 2^{12}$), while other approaches discuss video processing with techniques used for image and audio processing [165]. On the other hand, we do not find any complete multi-modal solution in the literature. We believe that as soon as there is a scalable implementation for image processing, this could be extended to be sufficiently general to process audio, video, and other types of data.

In Tables 2 and 3, we present the different implementations that we have reviewed for each dataset. We use Y as Yes, N as No, a question mark (?) if that characteristic is not disclosed in the paper, and a dash (—) to indicate that this characteristic does not apply to that work.

6 Performance

There are many dimensions for evaluating the performance of private NN inference, such as accuracy, latency, security, throughput, memory requirements, and energy-efficiency, among others. While latency (time-to-inference) is a primary concern in most papers, due to the additional overhead of homomorphic operations, it needs to be emphasized that there are trade-offs among accuracy, latency, and security. Per definition, CKKS works on approximate numbers and each operation introduces a (small) additional error that accumulates over many operations. Hence, a considerable amount of time in actual implementations is dedicated to error management techniques, restricting the error to a tolerable level. The definition of “tolerable” naturally depends on the application and some homomorphic operations (e.g. bootstrapping) represent a compromise between speed and accuracy. A less accurate bootstrap operation may be computed faster, but it also incurs larger error and even though NNs are known to be somewhat robust to reduced precision, there is no guarantee that a larger error will not change the inference outcome. Therefore, we consider it essential that authors report the accuracy of their encrypted inference in addition to the latency, especially if the reported latency indicates that this may be attained within a reasonable amount of time; otherwise, it is difficult to assess the significance of reported achievements, since these may arrive at the cost of reduced accuracy.

A similar argument may be made with regards to the security parameters. More secure parameters are generally associated with a higher computational cost and thus weakening the security requirements may lead to faster inference times. The specific parameters that may be considered sufficient are again application dependent, but 128-bit security seems to be established as the gold standard that most works in private inference aim to attain. Even though there have been attempts to standardize parameters across security levels [26], these standards should not be considered as something static, since the recommended parameters will change when new attack methods are developed. Despite this, not all papers in Tables 2, 3, and 4 that claim 128-bit security cite the corresponding source (i.e., the method to determine the security level) and some do not even disclose their encryption parameters. Thus, in some cases, it is impossible to

determine whether the 128-bit security claim (in those cases in which this feature is mentioned) did consider certain ways of attack (e.g., attacks benefiting from sparse keys [42]) or to re-evaluate their settings in the light of new attacks.

Furthermore, when comparing inference times (i.e., latency/throughput), it needs to be considered that the experimental setups (i.e., available hardware) differ among studies and may well contribute to reported differences in performance. Regardless of these impediments, we attempt to evaluate the performance of current private inference applications under three aspects that we consider critical in order for private inference to become practical.

6.1 Latency/Throughput

Despite the fact that sparse ciphertexts may be bootstrapped more efficiently than their dense counterparts, packing the ciphertexts as dense as possible (thus reducing their overall number) seems to be beneficial in terms of latency. This is demonstrated by multiplexed-packing [100], attaining $\approx 5\times$ faster ResNet-20 inference on a single thread compared to previous work [104] using one thread and (sparse) ciphertext per image channel. Subsequent works have built on this concept and reduced latency even further via novel packing strategies that minimize data layout rearrangements.

For ResNet-20 (the most commonly implemented network for the CIFAR-10 dataset), CPU inference time on a single thread has been reduced to slightly more than 10 minutes for classical CKKS slot encoding [49]. We note that although Rovida and Leporati [142] report a lower latency of only 5.5 minutes, we did not consider this a single-thread experiment, since the authors do not mention the number of threads explicitly and multi-threading is enabled per default in their underlying FHE library (OpenFHE). When benefiting from coefficient encoding to implement more efficient convolutions, Kim and Guyot [86] report a latency of roughly six minutes on a single thread.

When considering multi-threading on CPU, only HEaaN and OpenFHE support parallelization of their homomorphic operations and HEaaN currently seems to have the advantage in both the number of published works, as well as the reported latency. Park *et al.* [129] note that the ResNet-20 implementation of Lee *et al.* [100] may complete inference in three minutes using HEaaN and being executed on 64 threads. Further improving upon this, by replacing the activation function with AESPA and designing specialized convolution operations to minimize data rearrangements, HyPHEN [87] attains the lowest ResNet-20 inference time reported on CPU of 37.6 seconds.

On GPUs, this may be reduced even further, with reported latencies being as low as 1.4 seconds when using AESPA [87] or 1.67 seconds when using function approximation [88]. Even though implementations on other hardware architectures rarely test applications other than ResNet-20 on CIFAR-10, these seem to hold a significant advantage in terms of performance. FAB [3] is able to conduct inference in less than 0.27s on FPGAs and both SHARP [89] and Cinnamon [74] are able to reduce the time below 0.1s using ASICs.

Considering other NNs for the CIFAR-10 dataset (e.g. VGG-16), we observe a similar scenario, with the lowest latency being reported for the combination of HEaaN and GPU. For more complex datasets such as ImageNet, the most commonly implemented architectures are ResNet-18 and ResNet-50, and reported latencies range from almost 9 hours (CPU, single-threaded) down to 5.35 seconds on GPU. HyPHEN [87] shows the best performance on CPU (multi-threading, AESPA), but coefficient encoding also seems to be advantageous (although Kim & Guyot [86] only encrypt the final eight layers of their network). However, the lowest latency of 5.35 (with AESPA) and 14.11 seconds (with function approximation) on GPU is attained by the CinS encoding implemented in NeuJeans [81].

For transformer-based networks commonly employed in NLP, latencies range from less than two seconds to several hours, depending on the employed model and hardware architecture. Out of twelve implementations, eight employ GPUs to speed-up inference time, since pure CPU implementations seem to be far from practical. For example, MOAI [169], reports an amortized runtime on CPU of close to 10 minutes with a batch size of 256, resulting in a latency of more

than 40h. Similarly, Encrypted-LLM [46] note that, when switching to GPU, a speedup of $\approx 150X$ is observed, reducing inference time to ≈ 2 minutes. Despite these improvements, most works still report a latency of several minutes for transformer-based inference, even when using GPUs. Currently, specialized FHE accelerators seem to be the only means to reduce inference time even further, since Cinnamon [74] (the only ASIC tested for NLP) reports a latency of 1.67s.

In addition to latency, a few works also consider the amortized runtime for multiple inputs in the case of batched inference. For instance, since the ResNet-20 implementation of Lee et al. [100] only uses a single-thread, they suggest to use multiple-threads (one per input) for batched inference. Using 50 threads, they report a total latency of 4973 seconds, leading to an amortized latency of 79 seconds per image. Applying the same method to ResNet-32, they achieve an amortized latency of 127 seconds for the CIFAR-100 dataset. HELayers [4] proposes to optimizing the data layout considering different trade-offs, including batched versus single image processing. On SqueezeNet and CIFAR-10, they are able to achieve a speedup of almost 10X in amortized latency when comparing their batched optimizations to the single input ones. Consequently, some implementations optimize their data layout and algorithms specifically for batched inference [53, 66, 168, 169] and thus Tables 6,7,8,9 report the amortized latency and batch-size in those cases.

6.2 Accuracy

Besides efficiency of the execution, the ability of a model to solve a certain problem accurately is equally important. For classification tasks, this problem solving capability is usually measured via the accuracy (i.e., the number of correct classifications) for a certain dataset. Here, private inference via HE faces two challenges: 1) the cleartext model needs to be changed to facilitate homomorphic evaluation (e.g., activation function, average pooling) and 2) the additional error added during encryption, both of which might lead to a reduction in prediction accuracy. To address these issues and facilitate comparison, Podschwadt *et al.* [133] recommend that authors report both the state-of-the-art of a model (i.e., without any changes), as well as the cleartext and encrypted accuracy on the adapted model. Unfortunately, most papers report only the last two, which makes comparison more difficult, since both model changes and encryption parameters may well vary among different studies. Nonetheless, this section will attempt to provide a summary of the state-of-the-art for private inference accuracy, ordered by datasets.

For the CIFAR-10 dataset, the original paper [65] reports a ResNet-20 accuracy of approximately 92%, a number that is matched by most encrypted implementations. Orion [49] even reports a slightly higher number of 93.4%. Generally, most studies note that their encrypted inference results closely match the accuracy of the (adapted) cleartext model. Both activation function approximation and replacement seem to work equally well, since there are no notable differences. In terms of other models, ResNet-9 [116], ResNet-44 [87], and VGG-16 [49] are reported to attain more than 94% accuracy, and the highest accuracy of 98.3% is attained by ResNet-50 as implemented by Maloney *et al.* [116]. To the best of our knowledge, the state-of-the-art for this dataset is around 99.5%, attained by a transformer architecture [47].

When leveraging the CIFAR-100 dataset, significantly less results are reported. Lee *et al.* [100] note an accuracy of 69.43% for ResNet-32, Kim and Guyot [86] list 64.08% for ResNet-20, and the top accuracy is again reported by Maloney *et al.* [116], attaining 75.3% with a ResNet-9 architecture. This seems to be in line with unmodified versions of these architectures; it is, however, quite far from the state-of-the-art, which reaches more than 90% accuracy, even with convolution-based architectures [154]. One reason for this gap could be that, while CIFAR-100 increases the number of classes when compared to CIFAR-10, the dimensionality of the input remains unchanged, enabling private inference on both datasets with minimal changes to the network architecture. Thus, there seems to be less incentive to develop encrypted versions of networks specializing in CIFAR-100; many papers turn straight to ImageNet instead, which features both further classes and larger inputs.

The larger input size ($224 \times 224 \times 3$) and demand for deeper/larger networks increases the computational complexity, and single-threaded results for ImageNet are exceedingly rare. Kim and Guyot [86] report inference time of less than 5 minutes, but they evaluate only the final eight layers of ResNet-18 homomorphically, while the previous layers are implemented in cleartext. Orion [49] executes encrypted inference on the whole network, but due to the increased latency (more than 3h per image), do not provide accuracy results. Using a multi-threaded/GPU backend instead, HyPHEN [87] reports accuracy of 62.25% for the same network, while NeuJeans [81] manages to increase the accuracy up to 74.1% using ResNet-50. This number is matched by Maloney *et al.* [116], but when upscaling the images to $256 \times 256 \times 3$, they report a final accuracy of 80.2%. We note that the discussion on accuracy of ImageNet is limited to top-1 accuracy, since it is the most commonly reported number, but a few works (e.g., Kim and Guyot [86]) report top-5 accuracy instead.

To measure the accuracy of NLP tasks, most works employ the Glue benchmark [156], which consists of several datasets/tasks including sentiment analysis, question answering, and textual entailment. Most studies report private inference accuracies that are relatively close to the accuracies of the baseline model (usually within $\pm 1\%$).

6.3 Memory

Accurate memory consumption during private inference is only reported in a few papers and if no values are provided, Tables 6, 7, 8, and 9 list the available memory as an upper bound instead. Apart from the maximal ciphertext level (with a lower level needing less memory), there are two main components that may be tuned in order to decrease the memory demand: *a*) the evaluation keys and *b*) the plaintext weights. While the size of a single evaluation key is dictated by the homomorphic parameters (and largely depends on the maximal level), the number of required keys can be reduced at the cost of executing additional rotations. For example, any rotational offset could be attained by consecutively rotating the same ciphertext by a single slot (which would require one key only). As an indication, HyPHEN [87] notes that, in the ResNet-20 implementation by Lee *et al.* [100], a single evaluation key occupies more than 1GB of memory. In contrast, with their choice of parameters (and a lower level due to activation function replacement), a single key only needs 168 MB. In total, this reduces the memory requirements for all evaluation keys for ResNet-18 from 69.7 GB to 11.1 GB. On the other hand, Rovida and Leporati [142] pursue a different strategy, reducing the number of rotation keys needed during the convolution to a constant number (five). This enables them to conduct private inference on ResNet-20 with CIFAR-10 using less than 16GB of memory in total.

The size of the plaintext weights, on the other hand, inherently depend on the level at which the corresponding operation (i.e., the operation that applies the weights) is executed. The lower the level, the less memory is needed, which suggests that it may be possible to trade memory consumption for more frequent bootstrapping. Even though HELayers [4] considered optimizing data layouts according to memory requirements, to the best of our knowledge, none of the automated bootstrap placement strategies proposed so far has focused on optimizing memory. Especially for CNNs, the size of the plaintext weights are important, since common implementations (see Section 5.3.2) replicate each filter weight according to the size of the input. While this might be practical for small input data, it leads to an explosion of memory requirements for larger data such as ImageNet. During their attempt to enable private ImageNet inference on GPU, HyPHEN [87] reports that in common implementations, the filter plaintexts consume 285-365 GB of memory. Consequently, they propose an optimization called PRCP (“plaintext size reduction through the image to ciphertext rearrangement”), which essentially modifies the data layout to split an image into multiple segments. Hence, filter weights only needs to match the segment size, reducing the plaintext memory footprint to 45.6 GB.

Transformer-based architectures do not suffer from this weight replication problem and thus the evaluation keys remain the main memory constraint. Consequently, an A100 GPU with 80 GB of memory (which is commonly employed

in these works) does not experience any problems fitting the whole CKKS circuit and most papers do not mention the actual memory consumption. One exception is the study by Rovida and Leporati [143], who report a memory requirement of 18 GB for their implementation of the BERT-tiny model.

7 Related Work

In this section, we provide an overview of previous surveys and review papers on privacy-preserving machine learning (PPML). We analyze papers that discuss PPML solutions using HE or contrast it to other privacy-preserving technologies with the goal of achieving private inference. In Section 4, we justified why we believe that using a fully-HE solution is the most suitable option for a practical inference engine, supporting the focus of this work.

Combining HE and DNN is not an easy task and many surveys presenting DNN solutions with HE explore the challenges of constructing such approaches along with limitations and solutions to address these. For example, Pulido *et al.* [134] identified challenges on the computational design of the homomorphic processing of inner network functions, but also around data and error manipulation. They conclude that the objectives for NN inferencing leveraging HE are threefold: high accuracy, high data security, and low computational complexity. Their work analyzed early HE-based networks such as CryptoNets [57] and LoLa [29]. Along the same lines, Podschwadt *et al.* [133] reviewed the challenges of DNN architectures with HE, but reduced their references to publications that are well-known, practical and often tied to released software. They highlight difficulties in NN for HE such as computational, memory, and latency overheads, in addition to incompatible operations and limited network depth, while also including an overview of different FHE schemes and their advantages/disadvantages.

Hamza [63] highlighted open challenges and future directions on works using HE for AI-based applications together with a comparison between HE algorithms and libraries. They also discuss parallelization opportunities, and qualify the documentation of the solutions. Xu *et al.* [161] and Patel *et al.* [131] identified some future research directions in the field. The discussions in the latter on enhancing scalability, suggest parallelization efforts and exploring lightweight privacy-preserving methods suitable for resource-constrained environments. To reduce communication overhead, they propose to use model compression, sparsification, and aggregation methods, among others.

Other works discussed the **limitations on private inference**, along with potential solutions [8, 12]. The latter analyzes the major limitations identified in the literature, the most common approaches used to address those limitations, and identifies the most used types of NNs and the most commonly used HE schemes. Hong *et al.* [68] reviewed recent representative results of FHE-based PPML, discussing some results for inference that include NNs such as CryptoNets, TAPAS, SHE, general pre-trained models, tree-based models, ResNets, and transformers. The survey by Yuan *et al.* [165] focused on training and inference solutions based exclusively on CKKS, covering different types of data inputs until 2023; however, considerable progress since the writing of that article had not yet been discussed. There are studies focused on **inference and FHE**, aligning with our own focus, such as Pulido-Gaytan *et al.* [135], which discusses machine-learning as a service (MLaaS) for inference, with FHE as the tool for preserving privacy. However, that work is currently dated and it does not include CKKS implementations.

Other surveys covered solutions that use **other cryptographic primitives for privacy along with HE**. Aloufi *et al.* [11] discussed not only HE for two parties but they also consider multiple parties and other schemes including threshold HE, multi-key HE, or attribute-based encryption. Rechberger *et al.* [138] contrasted HE and MPC and the scenarios in which each of these may be used. They prescribed MLaaS as the suitable scenario for FHE, including training and inference tasks. They also discussed some of the limitations of the usage of FHE and MPC in machine-learning

protocols, such as being able to evaluate only linear functions. We note that they did not consider any other solution such as replacement to a HE-friendly function, which we discuss in Section 5.2.

Along the same lines, but focused on inference, there are surveys that **compared solutions that use MPC, FHE, and hybrid solutions**. We find in the literature a comparison and evaluation among CKKS, MPC and hybrid solutions of these two, limited to relatively small datasets [15]. That includes a discussion on the disadvantages of not being able to use non-linear layers. According to the paper, the CKKS scheme has the advantage of supporting batched computations. Similarly, Mann *et al.* [117] focused on secure NN inference solutions using techniques from HE and secure MPC, concretely, secret sharing, garbled circuits and, oblivious transfer. The focus of the work is on the inference phase and, as in our view, the requirements comprise low latency and high accuracy. In that work, the NN and the input are held by different parties and these have to remain private to the other party. Even though the focus of the paper is on solution techniques with strong cryptographic security, it does not identify special hardware requirements and there is no mention of large NN models or scalability. On the other hand, Li *et al.* [108] compared approaches that use two-party computation (2MP) techniques and HE, including hardware acceleration. However, that survey only reviews Cryptonets and PPCDNN [30], and it concludes that the best approach for secure prediction of NN is a hybrid solution between 2MP and HE, which we discarded in our discussion in Section 4. Li *et al.* [110] published a survey specific for private inference of transformers that considers not only, FHE but also MPC and functional encryption.

We intentionally leave out of the scope of our review of related work that is further away from our focus, such as those that discussed (F)HE schemes from a theoretical perspective, surveys focused on terminology definitions, or those around other privacy-preserving technologies such as differential privacy, trusted execution environments, federated learning, or functional encryption.

Compared to previous work, **our survey** focuses specifically on FHE applied to DNN inference of large models, including just-emerged solutions. In contrast, some surveys considered both HE and MPC, while many of the previous works did not consider bootstrapping, a basic ingredient to enable sufficient operations for DNN architectures.

8 Conclusion

In this work, we describe what we would consider to be a practical private inference engine and summarize the state-of-the-art of private inference and its applications. Although considerable improvements have been made in recent years, we conclude that there are still many challenges remaining. While bootstrapping has enabled private inference on deeper models, scalability remains a major concern. Practical applications are currently either restricted to models with a small number of parameters or need to be able to tolerate high latencies. We have concluded that CKKS is the most promising FHE scheme for encrypted inference on larger models; however, the cost of bootstrapping (in terms of latency) and accurate activation functions (in terms of level consumption) still limits many applications.

The fact that deeper NNs require more bootstrapping calls exacerbates the latency problem and implementations aim to find a good balance between bootstrapping frequency/placement and FHE parameters. Currently, many studies employ hand-crafted solutions to attain high performance, but this is only feasible for a small number of networks. For a general private inference engine, automated solutions need to be found to address this problem; otherwise, encrypted inference will remain restricted to certain architectures (e.g., ResNet) and thus be limited in scope. Similarly, polynomial approximation of the activation functions currently requires either a very high degree to attain sufficient accuracy, or it implies retraining the network. Ideally, we would like to find some solution with high accuracy and low latency produced with minimal fine-tuning. We believe that the emerging functional bootstrapping technique features the

potential to solve both problems, since the bootstrapping would also evaluate the activation function at the same time. However, the implications of using it apart from LUT precomputations remain to be studied.

Another observation that has to be made is that currently there is considerable literature volume focused on privacy-preserving CNNs, while the state-of-the-art in ML has transitioned to transformer-based architectures that outperform CNNs in many applications. Even though CNNs may provide sufficient accuracy for certain privacy-sensitive applications, widespread adoption of private inference will probably only be possible if it is able to mimic the solution quality of current ML applications.

We have also observed a gap in hardware acceleration techniques and open-source parallelism support (just OpenFHE among open-source libraries supports parallelism for CKKS). The existing research around ASIC accelerators has primarily used small ML models for benchmark/evaluation studies that bring into question their performance on large ML workloads like LLMs. Due to memory explosion in HE domain, cluster-based approaches, as proposed by Cinnamon, should be prioritized. We believe that the ideal inference engine has to combine techniques from software scalability and hardware acceleration to produce an entire system that is practical in production environments.

Acknowledgments

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them. This work is supported by ERC grant (HomeE, 101043467). Antonio J. Peña was in addition partially supported by the Ramón y Cajal fellowship RYC2020-030054-I funded by MCIN/AEI/ 10.13039/501100011033 and by “ESF Investing in your future”.

References

- [1] 2022. Lattigo v4. Online: <https://github.com/tuneinsight/lattigo>. EPFL-LDS, Tune Insight SA.
- [2] Rashmi Agrawal, Anantha Chandrakasan, and Ajay Joshi. 2024. HEAP: A Fully Homomorphic Encryption Accelerator with Parallelized Bootstrapping. In *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 756–769.
- [3] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. FAB: An FPGA-based Accelerator for Bootstrappable Fully Homomorphic Encryption. In *IEEE Int. Symp. on High-Performance Comp. Arch.*
- [4] E. Aharoni, A. Adir, M. Baruch, N. Drucker, G. Ezov, A. Farkash, L. Greenberg, R. Masalha, G. Moshkovich, D. Murik, H. Shaul, and O. Soceanu. 2023. HeLayers: A Tile Tensors Framework for Large Neural Networks on Encrypted Data. *Privacy Enhancing Technologies* 1, 325–342.
- [5] Ahmad Al Badawi, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, et al. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. In *10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*.
- [6] Ahmad Al Badawi, Louie Hoang, Chan Fook Mun, Kim Laine, and Khin Mi Mi Aung. 2020. PrivFT: Private and Fast Text Classification With Homomorphic Encryption. *IEEE Access* 8 (2020), 226544–226556.
- [7] Ahmad Al Badawi and Yuriy Polyakov. 2023. Demystifying Bootstrapping in Fully Homomorphic Encryption. *Cryptology Archive*, 2023/149 (2023).
- [8] Anmar A Al-Janabi, Sufyan T Faraj Al-Janabi, and Belal Al-Khateeb. 2023. Secure Data Computation Using Deep Learning and Homomorphic Encryption: A Survey. *International Journal of Online & Biomedical Engineering* 19, 11 (2023).
- [9] Martin R Albrecht. 2021. Lattice estimator documentation. <https://lattice-estimator.readthedocs.io/en/latest/>.
- [10] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. 2025. General Functional Bootstrapping Using CKKS. In *Advances in Cryptology (CRYPTO)*.
- [11] Asma Aloufi, Peizhao Hu, Yongsoo Song, and Kristin Lauter. 2020. Computing Blindfolded on Data Homomorphically Encrypted under Multiple Keys: An Extended Survey. *arXiv:2007.09270* (2020).
- [12] Ivone Amorim, Eva Maia, Pedro Barbosa, and Isabel Praça. 2023. Data Privacy with Homomorphic Encryption in Neural Networks Training and Inference. In *International Symposium on Distributed Computing and Artificial Intelligence*. Springer, 365–374.
- [13] Wei Ao and Vishnu Boddeti. 2025. CryptoFace: End-To-End Encrypted Face Recognition. In *Computer Vision and Pattern Recognition Conference*.
- [14] Wei Ao and Vishnu Naresh Boddeti. 2024. AutoFHE: Automated Adaption of CNNs for Efficient Evaluation over FHE. In *USENIX Security Symp.*
- [15] Monir Azraoui, Muhammad Bahram, Beyza Bozdemir, Sébastien Canard, Eleonora Ciceri, Orhan Ermis, Ramy Masalha, Marco Mosconi, Melek Önen, Marie Paindavoine, Boris Rozenberg, Bastien Vialla, and Sauro Vicini. 2020. *SoK: Cryptography for Neural Networks*. Springer, 63–81.
- [16] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. 2022. META-BTS: Bootstrapping Precision Beyond the Limit. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. 223–234.

- [17] Youngjin Bae, Jung Hee Cheon, Guillaume Hanrot, Jai Hyun Park, and Damien Stehlé. 2024. Plaintext-Ciphertext Matrix Multiplication and FHE Bootstrapping: Fast and Fused. In *Advances in Cryptology – CRYPTO 2024, Part III (Lecture Notes in Computer Science, Vol. 14922, 387–421)*.
- [18] Moran Baruch, Nir Drucker, Gilad Ezov, Yoav Goldberg, Eyal Kushnir, Jenny Lerner, Omri Soceanu, and Itamar Zimerman. 2024. Polynomial Adaptation of Large-Scale CNNs for Homomorphic Encryption-Based Secure Inference. In *Cyber Security, Cryptology, and Machine Learning*.
- [19] Moran Baruch, Nir Drucker, Lev Greenberg, and Guy Moshkovich. 2022. A Methodology for Training Homomorphic Encryption Friendly Neural Networks. In *Applied Cryptography and Network Security Workshops*. Springer International Publishing, 536–553.
- [20] Donald Beaver. 1991. Efficient Multiparty Protocols Using Circuit Randomization. In *Advances in Cryptology (CRYPTO)*.
- [21] Adrien Benamira, Tristan Guérard, Thomas Peyrin, and Sayandeep Saha. 2023. TT-TFHE: A Torus Fully Homomorphic Encryption-Friendly Neural Network Architecture. *arXiv:2302.01584* (2023).
- [22] Loris Bergerat, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. 2025. TFHE Gets Real: an Efficient and Flexible Homomorphic Floating-Point Arithmetic. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2025, 2 (2025), 126–162.
- [23] Fabian Boemer, Anamaria Costache, Rosario Cammarota, and Casimir Wierzynski. 2019. nGraph-HE2: A High-Throughput Framework for Neural Network Inference on Encrypted Data. In *7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC’19)*. 45–56.
- [24] Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe DM de Souza, and Vinodh Gopal. 2021. Intel HEXL: Accelerating Homomorphic Encryption with Intel AVX512-IFMA52. In *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. 57–62.
- [25] Fabian Boemer, Yixing Lao, Rosario Cammarota, and Casimir Wierzynski. 2019. nGraph-HE: A Graph Compiler for Deep Learning on Homomorphically Encrypted Data. In *Proceedings of the 16th ACM International Conference on Computing Frontiers*. 3–13.
- [26] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, et al. 2025. Security Guidelines for Implementing Homomorphic Encryption. *IACR Communications in Cryptology* 1, 4 (2025).
- [27] Jean-Philippe Bossuat, Christian Mouchet, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys. In *Advances in Cryptology (EUROCRYPT), Part I*. 587–617.
- [28] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. 2020. Chimera: Combining Ring-LWE-based Fully Homomorphic Encryption schemes. *Journal of Mathematical Cryptology* 14, 1 (2020), 316–338.
- [29] Alon Brutzkus, Oren Elisha, and Ran Gilad-Bachrach. 2018. Low Latency Privacy Preserving Inference. In *Int. Conference on Machine Learning*.
- [30] Hervé Chabanne, Amaury de Wargny, Jonathan Milgram, Constance Morel, and Emmanuel Prouff. 2017. Privacy-Preserving Classification on Deep Neural Network. *Cryptology ePrint Archive*, Report 2017/035.
- [31] J. Chao, A. Al Badawi, B. Unnikrishnan, J. Lin, C. F. Mun, J. M. Brown, J. P. Campbell, M. Chiang, J. Kalpathy, V. R. Chandrasekhar, P. Krishnaswamy, and K. M. M. Aung. 2019. CaREnets: Compact and Resource-Efficient CNN for Homomorphic Inference on Encrypted Medical Images.
- [32] Hao Chen, Ilaria Chillotti, and Yongsoo Song. 2019. Improved Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology*.
- [33] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT, Part I (Lecture Notes in Computer Science, Vol. 10820, 4–54)*. 360–384.
- [34] Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. 2025. SHIP: A Shallow and Highly Parallelizable CKKS Bootstrapping Algorithm. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 398–428.
- [35] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT 2017*, Tsuyoshi Takagi and Thomas Peyrin (Eds.). Springer International Publishing, Cham, 409–437.
- [36] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. 2020. Efficient Homomorphic Comparison Methods with Optimal Complexity. In *Advances in Cryptology – ASIACRYPT 2020 (Lecture Notes in Computer Science, Vol. 12492)*. Springer, 221–256.
- [37] Seonyoung Cheon, Yongwoo Lee, Dongkwan Kim, Ju Min Lee, Sunchul Jung, Taekyung Kim, Dongyoon Lee, and Hanjun Kim. 2024. DaCapo: Automatic Bootstrapping Management for Efficient Fully Homomorphic Encryption. In *USENIX Security 2024: 33rd USENIX Security Symposium*.
- [38] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *J. Cryptol.* 33, 1 (2020).
- [39] Ilaria Chillotti, Marc Joye, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. 2020. CONCRETE: Concrete Operates on Ciphertexts Rapidly by Extending TFHE. In *WAHC 2020–8th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, Vol. 15.
- [40] Ilaria Chillotti, Marc Joye, and Pascal Paillier. 2021. Programmable Bootstrapping Enables Efficient Homomorphic Inference of Deep Neural Networks. In *International Symposium on Cyber Security Cryptography and Machine Learning*. Springer, 1–19.
- [41] Mihai-Cristian Chindriș, Mihai Togan, and Ștefan-Ciprian Arseni. 2020. Secure Speaker Recognition System Using Homomorphic Encryption. In *International Conference on Information Technology and Communications Security*. Springer, 198–211.
- [42] Benjamin R Curtis and Rachel Player. 2019. On the Feasibility and Impact of Standardising Sparse-secret LWE Parameter Sets for Homomorphic Encryption. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. 1–10.
- [43] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: An Encrypted Vector Arithmetic Language and Compiler for Efficient Homomorphic Computation. In *ACM SIGPLAN Conf. on Programming Language Design and Implementation*.
- [44] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. 2019. CHET: An Optimizing Compiler for Fully-Homomorphic Neural-Network Inferencing. In *ACM SIGPLAN Conf. Programming Language Design Implementation*.
- [45] Leo de Castro, Rashmi Agrawal, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, Chiraag Juvekar, and Ajay Joshi. 2021. Does Fully Homomorphic Encryption Need Compute Acceleration? *Cryptology ePrint Archive*, Report 2021/1636.
- [46] Leo de Castro, Daniel Escudero, Adya Agrawal, Antigoni Polychroniadou, and Manuela Veloso. 2025. EncryptedLLM: Privacy-Preserving Large Language Model Inference via GPU-Accelerated Fully Homomorphic Encryption. In *Forty-second International Conference on Machine Learning*.

- [47] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *arXiv:2010.11929* (2020).
- [48] Jules Dumezy, Andreea Alexandru, Yuriy Polyakov, Pierre-Emmanuel Clet, Olive Chakraborty, and Aymen Boudguiga. 2025. Evaluating Larger Lookup Tables using CKKS. *Cryptology ePrint Archive*, Paper 2025/1301.
- [49] Austin Ebel, Karthik Garimella, and Brandon Reagen. 2025. Orion: A Fully Homomorphic Encryption Framework for Deep Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 734–749.
- [50] Austin Ebel and Brandon Reagen. 2024. Osiris: A Systolic Approach to Accelerating Fully Homomorphic Encryption. *arXiv:2408.09593* (2024).
- [51] Daniel L. Elworth and Sunwoong Kim. 2022. HEKWS: Privacy-Preserving Convolutional Neural Network-based Keyword Spotting with a Ciphertext Packing Technique. In *IEEE 24th International Workshop on Multimedia Signal Processing (MMSP)*. IEEE, 01–06.
- [52] G. Fan, M. Zhang, F. Zheng, S. Fan, T. Zhou, X. Deng, W. Tang, L. Kong, Y. Song, and S. Yan. 2025. WarpDrive: GPU-Based Fully Homomorphic Encryption Acceleration Leveraging Tensor and CUDA Cores. In *IEEE Int. Symp. on High Performance Computer Architecture (HPCA)*. 1187–1200.
- [53] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. 2023. TensorFHE: Achieving Practical Computation on Encrypted Data Using GPGPU. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 922–934.
- [54] K. Garimella, N. K. Jha, and B. Reagen. 2021. Sisyphus: A Cautionary Tale of Using Low-Degree Polynomial Activations in Privacy-Preserving Deep Learning. *arXiv:2107.12342* (2021).
- [55] Craig Gentry. 2009. *A Fully Homomorphic Encryption Scheme*. Ph. D. Dissertation. Stanford University.
- [56] Craig Gentry and Yongwoo Lee. 2025. Fully Homomorphic Encryption for Matrix Arithmetic. *Cryptology ePrint Archive*, Report 2025/1935.
- [57] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. 2016. CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy. In *Proceedings of The 33rd International Conference on Machine Learning*. PMLR, 201–210.
- [58] Robert E. Goldschmidt. 1964. Applications of division by convergence. In *Thesis (M.S.)—Massachusetts Institute of Technology, Dept. of Electrical Eng.*
- [59] A. Guimarães, E. Borin, and D. F. Aranha. 2021. Revisiting the Functional Bootstrap in TFHE. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* (2021).
- [60] Saransh Gupta, Rosario Cammarota, and Tajana Šimunić. 2024. MemFHE: End-to-End Computing with Fully Homomorphic Encryption in Memory. *ACM Transactions on Embedded Computing Systems* 23, 2 (2024), 1–23.
- [61] Shai Halevi and Victor Shoup. 2015. Bootstrapping for HELib. In *Advances in Cryptology (EUROCRYPT)*. 641–670.
- [62] Shai Halevi and Victor Shoup. 2018. Faster Homomorphic Linear Transformations in HELib. In *Advances in Cryptology (CRYPTO)*. 93–120.
- [63] Rafik Hamza. 2023. Homomorphic Encryption for AI-based Applications: Challenges and Opportunities. In *Int. Conf. on Knowledge & Systems Eng.*
- [64] Kyoohyung Han and Dohyeong Ki. 2020. Better Bootstrapping for Approximate Homomorphic Encryption. In *Topics in Cryptology (CT-RSA)*.
- [65] K. He, X. Zhang, S. Ren, and J. Sun. 2015. Deep Residual Learning for Image Recognition. *Conf. on Computer Vision and Pattern Recognition* (2015).
- [66] Z. He, M. Wang, X. Gao, Y. Luo, et al. 2025. ENSI: Efficient Non-Interactive Secure Inference for Large Language Models. *arXiv:2509.09424* (2025).
- [67] Ehsan Hesamifard, Daniel Takabi, and Mehdi Ghasemi. 2017. CryptoDL: Deep Neural Networks over Encrypted Data. *arXiv:1711.05189* (2017).
- [68] Cheng Hong. 2025. Recent Advances of Privacy-Preserving Machine Learning Based on (Fully) Homomorphic Encryption. *Security Safety* 4 (2025).
- [69] Edward J. Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*, Vol. 1. 3.
- [70] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. Cheetah: Lean and Fast Secure Two-Party Deep Neural Network Inference. In *31st USENIX Security Symposium*. 809–826.
- [71] Francesco Intoci, Sinem Sav, Apostolos Pyrgelis, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2023. slytHERin: An Agile Framework for Encrypted Deep Neural Network Inference. In *Applied Cryptography and Network Security Workshops*. 359–377.
- [72] Takumi Ishiyama, Takuya Suzuki, and Hayato Yamana. 2022. Latency-Aware Inference on Convolutional Neural Network Over Homomorphic Encryption. In *Information Integration and Web Intelligence*. Springer Nature Switzerland, 324–337.
- [73] Jaehye Jang, Younho Lee, Andrey Kim, Byunggook Na, Donggeon Yhee, Byoungan Lee, Jung Hee Cheon, and Sungroh Yoon. 2022. Privacy-Preserving Deep Sequential Model with Matrix Homomorphic Encryption. In *ACM Asia Conf. on Computer & Communications Security*. 377–391.
- [74] Siddharth Jayashankar, Edward Chen, Tom Tang, Wenting Zheng, and Dimitrios Skarlatos. 2025. Cinnamon: A Framework for Scale-Out Encrypted AI. In *30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS)*. 133–150.
- [75] Xiaoqian Jiang, Miran Kim, Kristin Lauter, and Yongsoo Song. 2018. Secure Outsourced Matrix Computation and Application to Neural Networks. In *ACM SIGSAC Conference on Computer and Communications Security (CCS’18)*. Association Computing Machinery, 1209–1222.
- [76] Dian Jiao, Xianglong Deng, Zhiwei Wang, Shengyu Fan, Yi Chen, Dan Meng, Rui Hou, and Mingzhe Zhang. 2025. Neo: Towards Efficient Fully Homomorphic Encryption Acceleration using Tensor Core. In *52nd Annual International Symposium on Computer Architecture*. 107–121.
- [77] Wenjie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and Wenguang Chen. 2025. BumbleBee: Secure Two-party Inference Framework for Large Transformers. In *ISOC Network and Distributed System Security Symposium – NDSS 2025*. The Internet Society.
- [78] Gilbert Jonatan, Haeyoon Cho, Hyojun Son, Xiangyu Wu, Neal Livesay, Evelio Mora, Kaustubh Shivdikar, José L. Abellán, Ajay Joshi, et al. 2024. Scalability Limitations of Processing-in-Memory Using Real System Evaluations. *Measurement and Analysis of Computing Systems* 8, 1 (2024).
- [79] Armand Joulin, Édouard Grave, Piotr Bojanowski, and Tomáš Mikolov. 2017. Bag of Tricks for Efficient Text Classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*. 427–431.
- [80] Marc Joye. 2021. Guide to Fully Homomorphic Encryption over the [Discretized] Torus. *Cryptology ePrint Archive*, Paper 2021/1402.
- [81] Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. 2024. NeuJeans: Private Neural Network Inference with Joint Optimization of Convolution and FHE Bootstrapping. In *Conference on Computer and Communications Security*.

- [82] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x Faster Bootstrapping in Fully Homomorphic Encryption through Memory-centric Optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems 2021* (2021).
- [83] Wonkyung Jung, Eojin Lee, Sangpyo Kim, Jongmin Kim, Namhoon Kim, Keewoo Lee, Chohong Min, Jung Hee Cheon, and Jung Ho Ahn. 2021. Accelerating Fully Homomorphic Encryption Through Architecture-Centric Analysis and Optimization. *IEEE Access* 9 (2021), 98772–98789.
- [84] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. 2018. GAZELLE: A Low Latency Framework for Secure Neural Network Inference. In *USENIX Security 2018: 27th USENIX Security Symposium*, USENIX Association (Ed.).
- [85] Andrey Kim, Maxim Deryabin, Jieun Eom, Rakyong Choi, Yongwoo Lee, Whan Ghang, and Donghoon Yoo. 2023. General Bootstrapping Approach for RLWE-Based Homomorphic Encryption. *IEEE Trans. Comput.* 73, 1 (2023), 86–96.
- [86] Dongwoo Kim and Cyril Guyot. 2023. Optimized Privacy-Preserving CNN Inference with Fully Homomorphic Encryption. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2175–2187.
- [87] Donghwan Kim, Jaiyoung Park, Jongmin Kim, Sangpyo Kim, and Jung Ho Ahn. 2024. HyPHEN: A Hybrid Packing Method and Its Optimizations for Homomorphic Encryption-Based Neural Networks. *IEEE Access* 12 (2024), 3024–3038.
- [88] J. Kim, W. Choi, and J. Ahn. 2024. Cheddar: A Swift Fully Homomorphic Encryption Library for CUDA GPUs. *arXiv:2407.13055* (2024).
- [89] Jongmin Kim, Sangpyo Kim, Jaewan Choi, Jaiyoung Park, Donghwan Kim, and Jung Ho Ahn. 2023. SHARP: A Short-Word Hierarchical Accelerator for Robust and Practical Fully Homomorphic Encryption. In *50th Annual International Symposium on Computer Architecture (ISCA)*.
- [90] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2023. ARK: Fully Homomorphic Encryption Accelerator with Runtime Data Generation and Inter-Operation Key Reuse. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
- [91] Jaehyung Kim and Taeyeon Noh. 2024. Modular Reduction in CKKS. *Cryptology ePrint Archive*, Paper 2024/1638.
- [92] Miran Kim, Xiaoqian Jiang, Kristin Lauter, Elkan Ismayilzada, and Shayan Shams. 2022. Secure Human Action Recognition by Encrypted Neural Network Inference. *Nature Communications* 13 (2022).
- [93] Sangpyo Kim, Jongmin Kim, Jaeyoung Choi, and Jung Ho Ahn. 2024. CiFHER: A Chiplet-Based FHE Accelerator with a Resizable Structure. In *Proceedings of the International Symposium on Secure and Private Execution Environment Design (SEED)*. 119–130.
- [94] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. BTS: An Accelerator for Bootstrappable Fully Homomorphic Encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 711–725.
- [95] Elena Kirshanova, Chiara Marcolla, and Sergi Rovira. 2024. Guidance for Efficient Selection of Secure Parameters for Fully Homomorphic Encryption. In *AFRICACRYPT 24: 15th International Conference on Cryptology in Africa, Lecture Notes in Computer Science, 14861*, 376–400. Springer.
- [96] Christian Knabenhans, Alexander Viand, Antonio Merino-Gallardo, and Anwar Hithnawi. 2023. vFHE: Verifiable Fully Homomorphic Encryption. In *Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. 11–22.
- [97] Aleksandar Krastev, Nikola Samardzic, Simon Langowski, Srinivas Devadas, and Daniel Sanchez. 2024. A Tensor Compiler with Automatic Data Packing for Simple and Efficient Fully Homomorphic Encryption. *Proc. ACM Program. Lang.* 8, PLDI, Article 152 (June 2024), 25 pages.
- [98] Clemens Krüger, Bhavinkumar Moriya, and Dominik Schoop. 2025. A Performance Comparison of the Homomorphic Encryption Schemes CKKS and TFHE. *Cryptology ePrint Archive*, Paper 2025/1460 (2025).
- [99] Brent Lagesse, Gabriel Nguyen, Utsav Goswami, and Kevin Wu. 2021. You Had to Be There: Private Video Sharing for Mobile Phones Using Fully Homomorphic Encryption. In *IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. 730–735.
- [100] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-Complexity Deep Convolutional Neural Networks on Fully Homomorphic Encryption using Multiplexed Parallel Convolutions. In *Int. Conf. on Machine Learning*.
- [101] Eunsang Lee, Joon-Woo Lee, Jong-Seon No, and Young-Sik Kim. 2020. Minimax Approximation of Sign Function by Composite Polynomial for Homomorphic Comparison. *Cryptology ePrint Archive*, Paper 2020/834.
- [102] Garam Lee, Minsoo Kim, Jai Hyun Park, Seung Won Hwang, and Jung Hee Cheon. 2022. Privacy-Preserving Text Classification on BERT Embeddings with Homomorphic Encryption. In *Conf. of the North American Chapter of the Association for Computational Linguistics (NAACL)*.
- [103] Junghyun Lee, Eunsang Lee, Joon-Woo Lee, Yongjune Kim, Young-Sik Kim, and Jong-Seon No. 2023. Precise Approximation of Convolutional Neural Networks for Homomorphically Encrypted Data. *IEEE Access* 11 (2023), 62062–62076.
- [104] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. 2022. Privacy-Preserving Machine Learning with Fully Homomorphic Encryption for Deep Neural Network. *IEEE Access* (2022).
- [105] Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. 2022. High-Precision Bootstrapping for Approximate Homomorphic Encryption by Error Variance Minimization. In *Int. Conf. on the Theory and Applications of Cryptographic Techniques*.
- [106] Dai Li, Akhil Pakala, and Kaiyuan Yang. 2022. MeNTT: A Compact and Efficient Processing-in-Memory Number Theoretic Transform (NTT) Accelerator. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 30, 5 (2022), 579–588.
- [107] Long Li, Jianxin Lai, Peng Yuan, Tianxiang Sui, Yan Liu, Qing Zhu, Xiaojing Zhang, Linjie Xiao, Wenguang Chen, and Jingling Xue. 2025. ANT-ACE: An FHE Compiler Framework for Automating Neural Network Inference. In *ACM/IEEE Int. Symp. on Code Generation and Optimization*.
- [108] M. Li, Y. Yan, Q. Wang, M. Du, Z. Qin, and C. Wang. 2021. Secure Prediction of Neural Network in the Cloud. *IEEE Network* 35 (2021), 251–257.
- [109] Shaohua Li, Kaiping Xue, Bin Zhu, Chenkai Ding, Xindi Gao, David Wei, and Tao Wan. 2020. FALCON: A Fourier Transform Based Approach for Fast and Secure Convolutional Neural Network Predictions. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [110] Yang Li, Xinyu Zhou, Yitong Wang, Liangxin Qian, and Jun Zhao. 2024. A Survey on Private Transformer Inference. *arXiv:2412.08145* (2024).
- [111] L. Lim et al. 2025. Tricycle: Private Transformer Inference with Tricyclic Encodings. *Cryptology ePrint*, 2025/1200 (2025).

- [112] Tzu-Li Liu, Yu-Te Ku, Ming-Chien Ho, Feng-Hao Liu, Ming-Ching Chang, Chih-Fan Hsu, Wei-Chao Chen, and Shih-Hao Hung. 2023. An Efficient CKKS-FHEW/TFHE Hybrid Encrypted Inference Framework. In *Computer Security: ESORICS International Workshops*. 535–551.
- [113] Guillermo Lloret-Talavera, Marc Jorda, Harald Servat, Fabian Boemer, Chetan Chauhan, Shigeki Tomishima, Nilesh N. Shah, and Antonio J. Peña. 2021. Enabling Homomorphically Encrypted Inference for Large DNN Models. *IEEE Trans. Comput.* 71, 5 (2021), 1145–1155.
- [114] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. In *Advances in Cryptology*.
- [115] Jiaao Ma, Ceyu Xu, and Lisa Wu Wills. 2023. PyTFHE: An End-to-End Compilation and Execution Framework for Fully Homomorphic Encryption Applications. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 24–34.
- [116] Vivian Maloney, Richard Obrecht, Vikram Saraph, Prathibha Rama, and Kate Tallaksen. 2024. High-Resolution Convolutional Neural Networks on Homomorphically Encrypted Data via Sharding Ciphertexts. *arXiv:2306.09189* (2024).
- [117] Zoltán Ádám Mann, Christian Weinert, Daphnee Chabal, and Joppe W. Bos. 2023. Towards Practical Secure Neural Network Inference: The Journey So Far and the Road Ahead. *ACM Comput. Surv.* 56, 5 (2023).
- [118] Junggho Moon, Zhanibek Omarov, Donghoon Yoo, Yongdae An, and Heewon Chung. 2024. Adaptive Successive Over-Relaxation Method for a Faster Iterative Approximation of Homomorphic Operations. *Cryptology ePrint Archive, Paper 2024/1366* (2024).
- [119] Junggho Moon, Dongwoo Yoo, Xiaoqian Jiang, and Miran Kim. 2025. THOR: Secure Transformer Inference with Homomorphic Encryption. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 3765–3779.
- [120] Mpoki Mwaesela, Joel Hari, Peterson Yuhala, Jāmes Ménétrey, Pascal Felber, and Valerio Schiavoni. 2024. Evaluating the Potential of in-Memory Processing to Accelerate Homomorphic Encryption: Practical Experience Report. In *43rd Int. Symp. on Reliable Distributed Systems (SRDS)*. 92–103.
- [121] L. Nguyen et al. 2025. A Novel Polynomial Activation for Audio Classification Using Homomorphic Encryption. *IEEE Access* 13 (2025), 87834–87847.
- [122] Nges Brian Njungle, Eric Jahns, and Michel A. Kinsy. 2025. FHEON: A Configurable Framework for Developing Privacy-Preserving Neural Networks Using Homomorphic Encryption. *arXiv:2510.03996* (2025).
- [123] Martin Nocker, David Drexel, Michael Rader, Alessio Montuoro, and Pascal Schöttle. 2023. HE-MAN—Homomorphically Encrypted Machine Learning with ONNX Models. *Proceedings of the 8th International Conference on Machine Learning Technologies* (2023).
- [124] Ali Sah Özcan and Erkey Savas. 2024. HEonGPU: a GPU-based Fully Homomorphic Encryption Library 1.0. *Cryptology ePrint Archive*, 2024/1543.
- [125] Marie Paindavoine and Bastien Vialla. 2016. Minimizing the Number of Bootstrappings in Fully Homomorphic Encryption. In *22nd Annual International Workshop on Selected Areas in Cryptography (SAC)*. 25–43.
- [126] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. 2024. BOLT: Privacy-Preserving, Accurate and Efficient Inference for Transformers. In *IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 4753–4771.
- [127] Dongjin Park, Eunsang Lee, and Joon-Woo Lee. 2025. Powerformer: Efficient and High-Accuracy Privacy-Preserving Language Model with Homomorphic Encryption. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, Vol. 1*. 11090–11111.
- [128] J. Park et al. 2022. AESPA: Accuracy Preserving Low-degree Polynomial Activation for Fast Private Inference. *arXiv:2201.06699* (2022).
- [129] Jaiyoung Park, Donghwan Kim, Jongmin Kim, Sangpyo Kim, Wonkyung Jung, Jung Hee Cheon, and Jung Ho Ahn. 2023. Toward Practical Privacy-Preserving Convolutional Neural Networks Exploiting Fully Homomorphic Encryption. *arXiv:2310.16530* (2023).
- [130] Jai Hyun Park. 2025. Ciphertext-Ciphertext Matrix Multiplication: Fast for Large Matrices. In *Advances in Cryptology (EUROCRYPT)*. 153–180.
- [131] Helly Patel, Ajay Patel, and Asha Patel. 2024. A Comprehensive Analysis of Privacy-Preserving Techniques in Machine Learning. In *1st International Conference on Advances in Computing, Communication and Networking (ICAC2N)*. IEEE, 1836–1841.
- [132] Robert Podschwadt and Daniel Takabi. 2021. Non-interactive Privacy Preserving Recurrent Neural Network Prediction with Homomorphic Encryption. In *IEEE 14th International Conference on Cloud Computing (CLOUD)*. IEEE, 65–70.
- [133] Robert Podschwadt, Daniel Takabi, Peizhao Hu, Mohammad H. Rafiei, and Zhipeng Cai. 2022. A Survey of Deep Learning Architectures for Privacy-Preserving Machine Learning with Fully Homomorphic Encryption. *IEEE Access* 10 (2022), 117477–117500.
- [134] Bernardo Pulido, Andrei Tchernykh, Jorge M. Cortés, Mikhail Babenko, Gleb Radchenko, Arutyun Avetisyan, and Alexander Yu Drozdov. 2021. Privacy-Preserving Neural Networks with Homomorphic Encryption: Challenges and opportunities. *Peer-to-Peer Netw. Appl.* 14, 3 (2021), 1666–1691.
- [135] Luis Bernardo Pulido-Gaytan, Andrei Tchernykh, Jorge M Cortés-Mendoza, Mikhail Babenko, and Gleb Radchenko. 2020. A Survey on Privacy-Preserving Machine Learning with Fully Homomorphic Encryption. In *Latin American High Performance Computing Conference*. Springer, 115–129.
- [136] Hongyuan Qu and Guangwu Xu. 2023. Improvements of Homomorphic Secure Evaluation of Inverse Square Root. In *Inf. & Com. Sec.* 110–127.
- [137] Michael O. Rabin. 2005. How To Exchange Secrets with Oblivious Transfer. *Cryptology ePrint Archive, Paper 2005/187*.
- [138] Christian Rechberger and Roman Walch. 2022. *Privacy-Preserving Machine Learning Using Cryptography*. Springer International, 109–129.
- [139] Oded Regev. 2009. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. *Journal of the ACM (JACM)* 56, 6 (2009), 1–40.
- [140] D. Rho, T. Kim, M. Park, J. W. Kim, H. Chae, E. K. Ryu, and J. H. Cheon. 2025. Encryption-Friendly LLM Architecture. In *Int. Conf. Learning Rep.*
- [141] M. Sadegh Riazzi, Kim Laine, Blake Pelton, and Wei Dai. 2020. HEAX: An Architecture for Computing on Encrypted Data. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [142] L. Rovida and A. Leporati. 2024. Encrypted Image Classification with Low Memory Footprint Using Fully Homomorphic Encryption. *International Journal of Neural Systems* (2024), 2450025–2450025.
- [143] L. Rovida and A. Leporati. 2024. Transformer-based Language Models and Homomorphic Encryption: An Intersection with BERT-Tiny. In *IWSPA'24: Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*. 3–13.
- [144] Arjun Roy and Kaushik Roy. 2024. DCT-CryptoNets: Scaling Private Inference in the Frequency Domain. In *Int. Conf. on Learning Representations*.

- [145] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
- [146] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. 2022. CraterLake: A Hardware Accelerator for Efficient Unbounded Computation on Encrypted Data. In *Int. Symp. Comp. Arch.*
- [147] Guilherme T. S. Sato and Wakaha Ogata. 2025. Privacy-Preserving Inference of Machine Learning Models without Retraining. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* advpub (2025), 2025CIP0011.
- [148] Shiyu Shen, Hao Yang, Yu Liu, Zhe Liu, and Yunlei Zhao. 2023. CARM: CUDA-Accelerated RNS Multiplication in Word-Wise Homomorphic Encryption Schemes for Internet of Things. In *IEEE Transactions on Computers*, Vol. 72. IEEE Computer Society, 1999–2010.
- [149] Menatallah Fadoua Slama, Hiba Guerrouache, Yacine Challal, Karima Benatchba, and Riyadh Baghdadi. 2025. Lightening Encrypted Convolutions: A GPU-Optimized Approach to Private Inference. In *IEEE Conference on Communications and Network Security (CNS)*. 1–9.
- [150] Samuel Sousa and Roman Kern. 2023. How to Keep Text Private? A Systematic Review of Deep Learning Methods for Privacy-Preserving Natural Language Processing. *Artificial Intelligence Review* 56, 2 (2023), 1427–1492.
- [151] Damien Stehlé, Ron Steinfield, Keisuke Tanaka, and Keita Xagawa. 2009. Efficient Public Key Encryption Based on Ideal Lattices. In *Adv. in Crypt.*
- [152] Andrei Stoian, Jordan Frery, Roman Bredehoft, Luis Montero, Celia Kherfallah, and Benoit Chevallier-Mames. 2023. Deep Neural Networks for Encrypted Inference with TFHE. In *Cyber Security, Cryptology, and Machine Learning*. Springer Nature Switzerland, Cham, 493–500.
- [153] Daniel Takabi et al. 2019. Privacy Preserving Neural Network Inference on Encrypted Data with GPUs. *arXiv:1911.11377* (2019).
- [154] Mingxing Tan and Quoc Le. 2021. EfficientNetV2: Smaller Models and Faster Training. In *Int. Conference on Machine Learning*. 10096–10106.
- [155] Alexander Viand, Patrick Jattke, and Anwar Hithnawi. 2021. SoK: Fully Homomorphic Encryption Compilers. In *IEEE Symp. on Security & Privacy*.
- [156] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. 353–355.
- [157] Ruida Wang, Jikang Bai, Xuan Shen, Xianhui Lu, Zhihao Li, Binwu Xiang, Zhiwei Wang, Hongyu Wang, Lutan Zhao, and Kumpeng Wang. 2025. Tetris: Versatile TFHE LUT and Its Application to FHE Instruction Set Architecture. *Cryptology ePrint Archive*, Paper 2025/1623.
- [158] Zeyu Wang and Makoto Ikeda. 2023. High-Throughput Privacy-Preserving GRU Network with Homomorphic Encryption. In *International Joint Conference on Neural Networks (IJCNN)*. 1–9.
- [159] Tommy White, Charles Gouert, Chengmo Yang, and Nektarios Georgios Tsoutsos. 2023. FHE-Booster: Accelerating Fully Homomorphic Execution with Fine-tuned Bootstrapping Scheduling. In *IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 293–303.
- [160] Wei Wu, Jian Liu, Huimei Wang, Fengyi Tang, and Ming Xian. 2018. PPolyNets: Achieving High Prediction Accuracy and Efficiency With Parametric Polynomial Activations. *IEEE Access* 6 (2018), 72814–72823.
- [161] R. Xu, N. Baracaldo, and J. Joshi. 2021. Privacy-Preserving Machine Learning: Methods, Challenges and Directions. *arXiv:2108.04417* (2021).
- [162] Zhihan Xu, Tian Ye, Rajgopal Kannan, and Viktor K. Prasanna. 2025. FAST: FPGA Acceleration of Fully Homomorphic Encryption with Efficient Bootstrapping. In *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25)*. 115–126.
- [163] Yinghao Yang, Huaizhi Zhang, Shengyu Fan, Hang Lu, Mingzhe Zhang, and Xiaowei Li. 2023. Poseidon: Practical Homomorphic Encryption Accelerator. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 870–881.
- [164] Andrew Chi-Chih Yao. 1982. Protocols for Secure Computations. In *Annual Symposium on Foundations of Computer Science (FOCS)*. 160–164.
- [165] Jiangjun Yuan, Weinan Liu, Jiawen Shi, and Qingqing Li. 2025. Approximate Homomorphic Encryption Based Privacy-Preserving Machine Learning: a Survey. *Artificial Intelligence Review* 58, 3 (2025), 82.
- [166] Zama. 2022. Concrete: TFHE Compiler that Converts Python Programs into FHE Equivalent. <https://github.com/zama-ai/concrete>.
- [167] Junxue Zhang, Xiaodian Cheng, Liu Yang, Jinbin Hu, Ximeng Liu, and Kai Chen. 2024. SoK: Fully Homomorphic Encryption Accelerators. In *ACM Computing Surveys*, Vol. 56. Association for Computing Machinery, New York, NY, USA.
- [168] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen-jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. 2025. Secure Transformer Inference Made Non-interactive. In *The Network and Distributed System Security (NDSS) Symposium*.
- [169] Linru Zhang, Xiangning Wang, Jun Jie Sim, Zhicong Huang, Jiahao Zhong, Huaxiong Wang, Pu Duan, and Kwok Yan Lam. 2025. MOAI: Module-Optimizing Architecture for Non-Interactive Secure Transformer Inference. *Cryptology ePrint Archive*, Paper 2025/991.
- [170] Peng Zhang, Ao Duan, Xianglu Zou, and Dongyan Qiu. 2026. Batch-Oriented Element-Wise Approximate Activation for Privacy-Preserving Neural Networks. In *Information and Communications Security*. 181–197.
- [171] Haoyun Zhu, Takuya Suzuki, and Hayato Yamana. 2023. Performance Comparison of Homomorphic Encrypted Convolutional Neural Network Inference Among HELib, Microsoft SEAL and OpenFHE. In *IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*.
- [172] Itamar Zimmerman, Moran Baruch, Nir Drucker, Gilad Ezov, Omri Soceanu, and Lior Wolf. 2024. Converting Transformers to Polynomial form for Secure Inference over Homomorphic Encryption. In *Forty-first International Conference on Machine Learning*.

Supplementary Material

Dataset	Architecture	Work	Inf. Engine	128-Bit Security	Library	Backend	Bootstrapping	Activation	Multi-threading	Retraining
CIFAR	AlexNet	DaCapo [37]	Y	Y	HEaaN	GPU	Y	App.	Y	N
		Orion [49]	Y	Y	Lattigo	CPU	Y	App.	N	N
	BERT	Zimmerman <i>et al.</i> [172]	N	Y	HEaan	GPU	Y	Rep.	Y	N
	ConvNeXt	Baruch <i>et al.</i> [18]	N	Y	HEaaN	C/GPU	Y	Rep.	Y	Y
	CNN	Ishiyama <i>et al.</i> [72]	N	Y	SEAL	CPU	N	Rep.	Y	Y
	DenseNet	Gui.&Wak. [147]	N	N	OpenFHE	CPU	Y	App.	Y	N
	LSTM	TensorFHE [53]	N	Y	—	GPU	Y	App.	Y	N
	MobileNet	DaCapo [37]	Y	Y	HEaaN	GPU	Y	App.	Y	N
		ANT-ACE [107]	Y	Y	—	CPU	Y	App.	?	N
	ResNet	AutoFHE [14]	N	Y	SEAL	CPU	Y	Rep.	Y	Y
		Baruch <i>et al.</i> [18]	N	Y	HEaaN	C/GPU	Y	Rep.	Y	Y
		Cheddar [88]	N	Y	—	GPU	Y	App.	Y	N
		DaCapo [37]	Y	Y	HEaaN	GPU	Y	App.	Y	N
		FAST [162]	N	Y	—	FPGA	Y	App.	-	N
		Fhelipe [97]	Y	Y	Lattigo	CPU	Y	App.	N	N
		FHEON [122]	N	N	OpenFHE	CPU	Y	App.	?	N
		HEAP [2]	N	Y	-	FPGA	Y	SSw.	—	N
		HyPHEN [87]	N	Y	HEaaN	C/GPU	Y	Rep.	Y	Y
		Kim&Guyot [86]	N	Y	Lattigo	CPU	Y	App.	N	N
		Lee <i>et al.</i> [104]	N	N	SEAL	CPU	Y	App.	Y	Y
		Lee <i>et al.</i> [100]	N	Y	SEAL	CPU	Y	App.	N	N
		Maloney <i>et al.</i> [116]	N	Y	OpenFHE	CPU	Y	Rep.	Y	Y
		NEO [76]	N	Y	—	GPU	Y	App.	Y	N
		Park <i>et al.</i> [129]	N	?	HEaaN	GPU	Y	Rep.	Y	Y
		Poseidon [163]	N	Y	—	FPGA	Y	App.	-	N
		Rov.&Lep. [142]	N	Y	OpenFHE	CPU	Y	App.	?	N
		Slama <i>et al.</i> [149]	N	?	HEonGPU	GPU	Y	App.	Y	N
		TensorFHE [53]	N	Y	—	GPU	Y	App.	Y	N
		WarpDrive [52]	N	Y	—	GPU	Y	App.	Y	N
	Squeeze Net	CHET [44]	Y	Y	SEAL	CPU	N	Rep.	Y	Y
		DaCapo [37]	Y	Y	HEaaN	GPU	Y	App.	Y	N
		EVA [43]	Y	Y	SEAL	CPU	N	Rep.	Y	Y
		HELayers [4]	Y	Y	SEAL	CPU	N	Rep.	Y	Y
		Zhang <i>et al.</i> [170]	N	Y	SEAL	CPU	N	Rep.	N	Y
	VGG	DaCapo [37]	Y	Y	HEaaN	GPU	Y	App.	Y	N
		Orion [49]	Y	Y	Lattigo	CPU	Y	App.	N	N
		AutoFHE [14]	N	Y	SEAL	CPU	Y	Rep.	Y	Y
		FHEON [122]	N	N	OpenFHE	CPU	Y	App.	?	N

Table 2. Characterization of CKKS Implementations - Part I. Inf. Engine stands Inference Engine, Rep. for replacement, App. is for polynomial approximation and SSw. is for Scheme Switching. CL in Backend is for CraterLake. FaceNet for CryptoFaceNet.

Dataset	Architecture	Work	Inf. Engine	128-Bit Security	Library	Backend	Bootstrapping	Activation	Multi-threading	Retraining
ImageNet	DenseNet	Gui.&Wak. [147]	N	Y	OpenFHE	CPU	Y	Approx.	Y	N
		Maloney <i>et al.</i> [116]	N	Y	OpenFHE	CPU	Y	Rep.	Y	Y
	MobileNet	NeuJeans [81]	N	Y	HEaaN	GPU	Y	Rep.	Y	Y
		Orion [49]	Y	Y	Lattigo	CPU	Y	App.	N	N
	ResNet	HyPHEN [87]	N	Y	HEaaN	C/GPU	Y	Rep.	Y	Y
		NeuJeans [81]	N	Y	HEaaN	GPU	Y	App.	Y	Y
		Orion [49]	Y	Y	Lattigo	CPU	Y	App.	N	N
		Park <i>et al.</i> [129]	N	?	HEaaN	GPU	Y	Rep.	Y	Y
COVIDx	AlexNet	HELayers [4]	Y	Y	SEAL	CPU	N	Rep.	Y	Y
WebFace4M	FaceNet	CryptoFace [13]	N	Y	SEAL	CPU	Y	Rep.	Y	Y
IMDB	PrivFT	Al Badawi [6]	N	N	SEAL	C/GPU	N	App.	Y	Y
	RNN	Fhelipe [97]	Y	Y	Lattigo	CL	Y	App.	N	N
		Pods.&Tak. [132]	Y	Y	Helib	CPU	Y	Rep.	Y	Y
AGNews	GRU	Wang&Ikeda [158]		Y	HEaan	CPU	N	App.	N	N
	PrivFT	Al Badawi [6]	N	N	SEAL	C/GPU	N	App.	Y	Y
THUCNews	~ BERT	EMMFormer	Y	?	?	C/GPU	N	App.	Y	N
SogouNews										
Yelp	PrivFT	Al Badawi <i>et al.</i> [6]	N	N	SEAL	C/GPU	N	App.	Y	Y
DBPedia										
WikiText103	BERT	Zimerman [172]	N	Y	HEaan	CPU	Y	Rep.	Y	N
YTSC	PrivFT	Al Badawi <i>et al.</i> [6]	N	N	SEAL	C/GPU	N	App.	Y	Y
	BERT	Lee <i>et al.</i> [102]	N	Y	?	GPU	Y	App.	Y	N
EED	PrivFT	Al Badawi <i>et al.</i> [6]	N	N	SEAL	C/GPU	N	App.	Y	Y
Amazon	RNN	Pods.&Tak. [132]	Y	Y	Helib	CPU	Y	Rep.	Y	Y
Twitter	BERT	Lee <i>et al.</i> [102]	N	Y	?	GPU	Y	App.	Y	N
SNIPS										
RTE	BERT	NEXUS [168]	Y ^a	Y	SEAL	CPU	Y	App.	Y	N
		MOAI [169]	Y	Y	SEAL	CPU	Y	App.	N	Y
		Rho <i>et al.</i> [140]	N	Y	HEaaN	GPU	Y	Rep.	N	Y
		THOR [119]	N	Y	Liberate	GPU	Y	App.	N	Y
SST	BERT-t	Powerformer [127]	Y ^b	Y	Liberate	CPU	Y	Rep.	N	Y
	Llama ^c	ENSI [66]	N	Y	OpenFHE	C/GPU	Y	App.	N	N
	BERT-t	Tricycle [111]	N	Y	OpenFHE	CPU	Y	App.	Y	N
SST-2	BERT	NEXUS [168]	Y ^a	Y	SEAL	CPU	Y	App.	Y	N
		MOAI [169]	Y	Y	SEAL	CPU	Y	App.	N	Y
		Rho <i>et al.</i> [140]	N	Y	HEaaN	GPU	Y	Rep.	N	Y
		THOR [119]	N	Y	Liberate	GPU	Y	App.	N	Y
	BERT-t	Powerformer [127]	Y ^b	Y	Liberate	CPU	Y	Rep.	N	Y
		Rov.&Lep. [143]	N	Y	OpenFHE	CPU	Y	App.	N	N
	GPT-2	EncLLM [46]	N	Y	OpenFHE	C/GPU	Y	Rep.	N	N
QNLI	BERT	MOAI [169]	Y	Y	SEAL	CPU	Y	App.	N	Y
		Rho <i>et al.</i> [140]	N	Y	HEaaN	GPU	Y	Rep.	N	Y
	Llama ^d	NEXUS [168]	Y ^a	Y	Phantom	GPU	Y	App.	Y	N

Table 3. Characterization of CKKS Implementations - Part II. Liberate stands for LiberateFHE, BERT-t for BERT-tiny, EncLLM for EncryptedLLM. ^aWith limit token size (128). ^bCan be extended to BERT-medium and BERT-base. ^cLlama-3-700M. ^dLlama-3-8B.

Dataset	Architecture	Work	Inf. Engine	128-Bit Security	Library	Backend	Bootstrapping	Activation	Multi-threading	Retraining
MRPC	BERT-t	Powerformer [127]	Y ^b	Y	Liberate	CPU	Y	Rep.	N	Y
	BERT	Rho <i>et al.</i> [140]	N	Y	HEaaN	GPU	Y	Rep.	N	Y
		THOR [119]	N	Y	Liberate	GPU	Y	App.	N	Y
HellaSwag	GPT-2	EncLLM [46]	N	Y	OpenFHE	C/GPU	Y	Rep.	N	N
ARC										
Social IQa										
MNLI										
ANLI										
WiC										
PIQA	Llama ^c	ENSI [66]	N	Y	OpenFHE	C/GPU	Y	App.	N	N
COPA										
CoLA	BERT	Rho <i>et al.</i> [140]	N	Y	HEaaN	GPU	Y	Rep.	N	Y
STSB										
Google's Speech Com.	HEKWS	Hekws [51]	N	Y	SEAL	CPU	N	Rep.	N	N
Speaker ^e	CNN	Chindriş <i>et al.</i> [41]	N	?	SEAL	CPU	N	Rep.	N	N

Table 4. Characterization of CKKS Implementations - Part III. ^bCan be extended to BERT-medium and BERT-base. ^cLlamaA-3-700M. ^e5242 spectral samples of 13 speakers.

Work	128-bit secure	Backend	Architecture	Dataset	Compiler/DSL	Word-size	Memory (MB)	Main Focus	Frequency (GHz)	Power (W)	Area (mm ²)
Poseidon [163]	Y	FPGA	ResNet20	CIFAR10	-	32	43	Bootstrap	0.45	-	512 PEs
HEAP [2]	Y	FPGA	ResNet20	CIFAR10	-	54	43	HSSw	0.3	-	512 PEs
FAST [162]	Y	FPGA	ResNet20	CIFAR10	-	32	43	Bootstrap	0.3	-	512 PEs
F1 [145]	Y	ASIC	LoLA	CIFAR10	Y	32	64	Flexible	1	180.4	151.4
CraterLake [146]	Y	ASIC	ResNet-20 LSTM	CIFAR10 IMDB	Y	28	256	Keyswitch	1	320	472
BTS [94]	Y	ASIC	ResNet20	CIFAR10	N	64	512	Bootstrap	1.2	163.2	373.6
ARK [90]	Y	ASIC	ResNet20	CIFAR10	N	64	512	Bootstrap	1	281.3	418.3
SHARP [89]	Y	ASIC	ResNet20	CIFAR10	N	36	180	Bit width	1	98	178.8
Osiris [50]	Y	ASIC	ResNet20	CIFAR10	N	40	210	Dataflow	1	119	246.9
CiFHER [93]	Y	ASIC	ResNet20	CIFAR10	N	32	256	Chiplet	1	~120	310.59
Cinnamon [74]	Y	ASIC	ResNet20 BERT-base	CIFAR10	Y	28	56	Cluster	1	190 /chip	223.18 /chip

Table 5. Hardware acceleration solutions. PE stands for processing element.

Dataset	Architecture	Work	Backend	Latency (s)	Memory (GB)	Accuracy (%)
CIFAR-10	SqueezeNet	CHET [44]	CPU	164.7	≤ 224	79.31 ^a
		EVA [25]	CPU	72.2	≤ 190	79.34
		HELayers [4]	CPU	131.8	≤ 750	?
		Zhang <i>et al.</i> [170]	CPU	131.8	≤ 750	?
	ResNet-9	Maloney <i>et al.</i> [116]	CPU	3131.5*	≤ 256	85.17
	VGG-11	AutoFHE [14]	CPU	1515*	≤ 768	86.78
		FHEON [122]	CPU	1647	≤ 64	86.10
	VGG-16	DaCapo [37]	GPU	~ 40	?	93.8
		FHEON [122]	CPU	2232	≤ 64	85.80
		Orion [49]	CPU	588.6	≤ 512	94.50
		ANT-ACE [107]	CPU	1454	≤ 512	91.20
		ARK [90]	ASIC	0.125	?	?
		AutoFHE [14]	CPU	4295*	≤ 768	91.96
		BTS [94]	ASIC	1.91	?	?
		Cheddar [88]	GPU	0.82	80	91.88
		CiFHER [93]	ASIC	0.222	?	?
		Cinnamon [74]	ASIC	0.070	?	?
		CraterLake (CL) [146]	ASIC	0.249	?	?
		DaCapo [37]	GPU	~ 20	?	90.7
		FAST [162]	FPGA	1.859	?	?
		Fhelipe [97]	CPU	1555	?	91.00
			CL	0.2	?	91.00
		FHEON [122]	CPU	403	20.4	92.20
	ResNet-20	HEAP [2]	FPGA	0.267	?	94
		HyPHEN [87]	CPU	37.6	≤ 480	92.17
			GPU	1.4	≤ 80	92.17
		Kim&Guyot [86]	CPU	368	≤ 100	90.32
		Lee <i>et al.</i> [104]	CPU	10602	172	92.43
		Lee <i>et al.</i> [100]	CPU	2271	≤ 512	91.31
		NEO [76]	GPU	9.11	≤ 80	?
		Orion [49]	CPU	618.2	≤ 512	93.40
		Osiris [50]	ASIC	0.963	?	?
		Park <i>et al.</i> [129]	GPU	1.4	≤ 80	92.18
		Poseidon [163]	FPGA	2.661	?	?
		Rov.&Lep. [142]	CPU	336	~ 15.2	91.67
		SHARP [89]	ASIC	0.099	?	?
		Slama <i>et al.</i> [149]	GPU	29.8	≤ 160	91.97
		TensorFHE [53]	GPU	316.1*	≤ 80	?
		WarpDrive [52]	GPU	5.88	≤ 80	?
	ResNet-44	ANT-ACE [107]	CPU	3054	≤ 512	90.70
		AutoFHE [14]	CPU	8078*	≤ 768	92.55
		HyPHEN [87]	CPU	78.0	≤ 480	94.08
			GPU	3.0	≤ 80	94.08
	ResNet-50 ^b	Baruch <i>et al.</i> [18]	CPU	9129	173.37	93.58 ^c
			GPU	1862	173.37	93.58 ^c

Table 6. Performance time and message exchange for executing one secure inference over the specified dataset - Part I. *Batched inference. ^aFrom [25]. ^bInput resized to $224 \times 224 \times 3$. ^cAccuracy for HE-friendly, unencrypted model without skip-connections.

Dataset	Architecture	Work	Backend	Latency (s)	Memory (GB)	Accuracy (%)
CIFAR-10	ResNet-50	Maloney <i>et al.</i> [116]	CPU	4321	≤ 800	98.3
		ANT-ACE [107]	CPU	3908	≤ 512	93.80
	ResNet-56	NEO [76]	GPU	26.48	≤ 80	?
		Lee <i>et al.</i> [100]	CPU	6852	≤ 512	93.07
	CNN	Ishiyama <i>et al.</i> [72]	CPU	12.9	255.4	80.96
	LoLA	F1 [145]	ASIC	0.241	?	?
CIFAR-100	DenseNet	Gui.&Wak. [147]	CPU	19192*	?	95.57
	ResNet-9	Maloney <i>et al.</i> [116]	CPU	942	≤ 128	75.3
	VGG11	AutoFHE [14]	CPU	4562*	≤ 768	64.31
	ResNet-20	Kim&Guyot [86]	CPU	368	≤ 100	64.08
	ResNet-32	AutoFHE [14]	CPU	10969*	≤ 768	71.34
		Lee <i>et al.</i> [100]	CPU	3942	≤ 512	69.43
ImageNet	ResNet-34	FHEON [122]	CPU	1594	≤ 64	74.4
	ResNet-18 ^e	Kim&Guyot [86]	CPU	292	≤ 100	88.05 ^f
			CPU	373.4	56.7	65.25
		HyPHEN [87]	GPU	14.7	56.7	65.25
	ResNet-18			14.11		64.8
		NeuJeans [81]	GPU	5.35	≤ 80	66.4 ^d
		Park <i>et al.</i> [129]	GPU	14.7	?	69.78
	ResNet-34	Orion [49]	CPU	14338	≤ 512	?
	ResNet-50	Baruch <i>et al.</i> [18]	GPU	1862	173.37	76.20 ^k
		Maloney <i>et al.</i> [116]	CPU	13009	≤ 800	80.2
	ResNet-50	NeuJeans [81]	GPU	56.1	≤ 80	74.1
		Orion [49]	CPU	32324	≤ 512	?
COVIDx	DenseNet	Gui.&Wak. [147]	CPU	32169	?	76.75
WebFace4M	AlexNet	HELayers [4]	CPU	298.8	163	80.60
YTSC	CryptoFaceNet16	CryptoFace [13]	CPU	1446	277	91.46
	PrivFT	Al Badawi <i>et al.</i> [6]	GPU	5 days	?	86.3 ^j
IMDB	SentenceBERT	Lee <i>et al.</i> [102]	GPU	460.81	?	90.8
	RNN	Fhelipe [97]	CPU	4502	?	78
			CL	0.4	?	78
			CPU	14 text/s ⁱ	?	72.99 ^g
	LSTM	CraterLake [146]	ASIC	0.138	?	?
			GPU	123.1*	≤ 80	?
AGNews	PrivFT	Al Badawi <i>et al.</i> [6]	GPU	< 0.66	390 MB	91.49
	GRU	Wang&Ikeda [158]	CPU	90 text/h	128	92.54
Twitter SNIPS	SentenceBERT	Lee <i>et al.</i> [102]	GPU	?	?	98.79 ^g /99.58 ^h
Amazon						98.76 ^g /99.89 ^h
DBPedia	RNN	Pods.&Tak. [132]	CPU	14 text/s ⁱ	?	86.31 ^g
Yelp	PrivFT	Al Badawi <i>et al.</i> [6]	GPU	< 0.66	390 MB	98.80
						96.06

Table 7. Performance time and message exchange for executing one secure inference over the specified dataset - Part II. *Batched inference. ^dAESPA. ^eFinal 8 layers. ^fTop-5. ^gF1 metric, ^hAUC metric. ⁱ1179 seconds for 16k instances. ^jExperiment conducted by Lee *et al.* ^kAccuracy for HE-friendly unencrypted model.

Dataset	Architecture	Work	Backend	Embed. dim.	Tokens	Latency (s)	Memory (GB)	Accuracy Glue (%)	
WikiText103				768	128	305	≤ 80	?	
ViT	BERT (6l)	Zimmerman <i>et al.</i> [172]	C/GPU	192	128	305	≤ 80	?	
Swin				96	128	305	≤ 80	?	
RTE	BERT-base	NEXUS [168]	CPU	?	128	857*	≤128	69.88	
			GPU	?	8	37.34*	≤ 160	69.88	
		THOR [119]	GPU	?	128	602.26	≤ 80	71.12	
	BERT	MOAI [169]	CPU	768	128	574.6*	?	65.5	
		Rho <i>et al.</i> [140]	GPU	768	128	26.5	≤ 192	60	
	Llama-3-8B	NEXUS [168]	CPU	?	128	1088*	≤128	81.24	
			GPU	?	8	51.84	≤ 160	81.24	
		LlamaA ^l	ENSI [66]	C/GPU	?	2048	958.12*	≤ 80	55.08
SST	BERT-tiny	Tricycle [111]	CPU	?	13	~ 450	≤ 4 TB	?	
				?	40	~ 1000	≤ 4 TB	?	
SST-2	BERT-tiny	Rov.&Lep. [143]	CPU	20	?	~375	~18	-	
				40	?	~600	~18	?	
	BERT-base	NEXUS [168]	CPU	?	128	857*	≤128	92.11	
			GPU	?	8	37.34*	≤160	92.11	
	THOR [119]	GPU	?	128	602.26	≤ 80	90.71		
	BERT	Rho <i>et al.</i> [140]	GPU	768	128	26.5	≤192	82	
		MOAI [169]	CPU	768	128	574.6*	?	93.8	
	Llama-3-8B	NEXUS [168]	CPU	?	128	1088*	≤128	94.46	
			GPU	?	8	51.84*	≤160	94.46	
	GPT2-small	EncLLM [46]	GPU	768	128	~ 10 ²	≤80	56	
	QNLI	BERT-base	NEXUS [168]	CPU	?	128	857*	≤128	89.90
				GPU	?	8	37.34*	≤160	89.90
		BERT	MOAI [169]	CPU	768	128	574.6*	?	90.5
Rho <i>et al.</i> [140]			GPU	768	128	26.5	≤192	78	
Llama-3-8B		NEXUS [168]	CPU	?	128	1088*	≤128	90.20	
			GPU	?	8	51.84*	≤160	90.20	
MRPC		BERT	Rho <i>et al.</i> [140]	GPU	768	128	26.5	≤192	81
		BERT-base	THOR [119]	GPU	?	128	602.26	≤80	84.8/89.49 ^g
	BERT-tiny	Powerformer [127]	CPU	?	?	503	?	?	
HellaSwag								28	
ARC								43	
Social IQa								37	
MNLI	GPT2-small	EncLLM [46]	GPU	768	128	~ 10 ²	≤80	33	
ANLI								35	
WiC								51	
PIQA									
COPA								LlamaA ^l	ENSI [66]
CoLA	BERT (2l)	Rho <i>et al.</i> [140]	GPU	768	128	26.5	≤192	16	
STS	BERT	Rho <i>et al.</i> [140]	GPU	768	128	26.5	≤192	80	
THUCNews	~ BERT	EMMFormer	C/GPU	300	?	~ 24000	≤ 339	83.17	
Sogou News						~ 36000	≤ 339	~ 80	

Table 8. Performance time and message exchange for executing one secure inference over the specified dataset - Part III. *Batched inference. Accuracy measured by Glue benchmark unless ^g for F1 metric or ^h for AUC metric. ^lLlamA-3-700M.

Dataset	Architecture	Work	Backend	Latency (s)	Memory (GB)	Accuracy Glue (%)
?	BERT-base	Cinnamon [74]	ASIC	1.67	?	?
Google's Speech Commands	HEKWS	Hekws [51]	CPU	0.0185 s/keyword	≤ 128	72.3
Speaker samples ^m	7layer CNN	Chindriş <i>et al.</i> [41]	CPU	~ 12.18	≤ 100	99.99
audioMNIST	5layer CNN	Nguyen <i>et al.</i> [121]	CPU	?	≤ 139	~ 95
Musical instrument						~ 97

Table 9. Performance time and message exchange for executing one secure inference over the specified dataset - Part IV. ^m5242 spectral samples of 13 speakers.